

Servidor MAGINOT - Anexo I

Dr. Cristian Cervantes

2026-04-26

Table of contents

Preface	6
Alcance de la bitácora	6
Ficha técnica del servidor MAGINOT	6
CPU	7
Memoria RAM	7
GPU	7
Almacenamiento y puntos de montaje (XFS)	7
Red (resumen)	8
Arquitectura de red y acceso	8
1 Introducción	10
1.1 Objetivo principal	10
1.2 Objetivos específicos	10
2 Validación y configuración de discos	12
2.1 Contexto y objetivo	12
2.1.1 Aclaración sobre el cambio de estrategia (RAID1 → volúmenes individuales)	12
2.2 Inventario y detección de dispositivos	12
2.2.1 Sobre los nombres <code>/dev/sdX</code>	13
2.3 Estado actual (volúmenes individuales integrados)	13
2.4 Procedimiento estándar (molde) para agregar discos nuevos sin RAID	14
2.4.1 Identificar el disco por MODEL/SERIAL y/o <code>by-id</code>	14
2.4.2 Desmontar si estaba montado	14
2.4.3 Borrar firmas y tabla de particiones	14
2.4.4 Crear GPT + 1 partición (100%)	15
2.4.5 Formatear a XFS con etiqueta	15
2.4.6 Crear punto de montaje y montar	15
2.4.7 Agregar a <code>/etc/fstab</code> y probar	15
2.4.8 Verificación final (rápida)	16
3 VPN con WireGuard	17
3.1 Instalación de WireGuard	17
3.2 Cargar el módulo del kernel y verificar	17
3.3 Identificar la interfaz de salida (NAT)	18
3.4 Crear llaves del servidor	18

3.5	Crear la interfaz <code>wg0</code> (configuración del servidor)	18
3.6	Habilitar IP forwarding	19
3.7	Abrir el puerto 13420/udp en firewall	19
3.8	Activar WireGuard (<code>wg-quick@wg0</code>)	19
3.9	Estado de WireGuard	20
3.10	Crear clientes por nombre	20
3.10.1	Script de alta: <code>wg-add-client</code> (crea peer + archivo <code>NOMBRE.conf</code>)	20
3.10.2	Instalar el script	21
3.10.3	Crear un cliente (por nombre)	22
3.11	Copiar el <code>.conf</code> a un home (para transferencia a teléfono/PC)	23
3.12	Revocar un cliente (baja)	23
4	Fortalecimiento de Acceso y Autenticación	24
4.1	Contraseñas	24
4.1.1	Contexto y objetivo	24
4.1.2	Requisitos mínimos de contraseña	24
4.2	Definir la política en <code>/etc/security/pwquality.conf</code>	25
4.2.1	Verificación inmediata (para confirmar que ya aplica)	25
4.2.2	Crear el script <code>/usr/local/sbin/maginot-add-user</code>	26
4.2.3	Uso del script (alta de usuarios)	29
4.2.4	Mensajes de política (una sola vez)	30
4.3	Configuración de SSH	30
4.3.1	Contexto y objetivo	30
4.3.2	Cambios en <code>/etc/ssh/sshd_config</code>	30
4.4	Activar Fail2Ban	31
5	Contenedores y Entornos Reproducibles	33
5.1	Contexto y objetivo	33
5.2	Estado actual: Docker y Podman instalados.	34
5.2.1	Versiones y verificación	34
5.2.2	Docker (prueba rápida, sin construir)	34
5.2.3	Podman (prueba rápida, sin construir)	35
5.3	Apptainer (+ fakerooot)	35
5.3.1	Verifica que ya existe el comando	35
5.3.2	Prueba rápida (sin construir nada)	36
5.4	Micromamba	36
5.4.1	Descargar el binario e instalarlo en <code>/usr/local/bin</code> :	36
5.4.2	Verificación	36
5.4.3	Prefijo global (ambientes compartidos) y permisos “solo root escribe”	37
5.4.4	Configuración global (canales y prioridades)	37
5.4.5	Habilitar para todos los usuarios (activate/run) vía <code>/etc/profile.d</code>	37
5.4.6	Aplicar en sesión actual (o cerrar sesión y re-entrar)	38
5.4.7	Crear ambientes e instalar paquetes (solo como root)	38

5.4.8	Usar el ambiente desde un script	38
5.4.9	Reproducibilidad (lock del ambiente)	38
5.5	Multiversión de software (Lmod / Environment Modules)	39
5.5.1	Instalar Lmod en Rocky (EPEL + CRB)	39
5.5.2	Asegurar que <code>module</code> funcione para <i>todos</i> (login y no-login)	39
5.5.3	Árbol local de modulefiles y permisos	40
5.5.4	Hacer que Lmod <i>siempre</i> vea <code>/opt/modulefiles</code>	40
5.5.5	Publicar “programas” a usuarios mediante módulos	40
5.5.6	Apptainer (<code>.sif</code>) + Lmod (envolver comando)	41
5.5.7	Nota importante: evita conflictos con <code>environment-modules</code>	42
5.6	Buenas prácticas de reproducibilidad	43
5.6.1	Checklist final (uso por usuarios)	43
6	Servidor de archivos con Samba	46
6.1	Contexto y objetivo	46
6.2	Alcance de la configuración	46
6.3	Preparación de la carpeta compartida	46
6.3.1	Establecer propietario	47
6.3.2	Restringir permisos	47
6.3.3	Instalación de Samba	47
6.4	Configuración de SELinux	47
6.4.1	Instalar <code>semanage</code> si es necesario	47
6.4.2	Asignar contexto SELinux	47
6.5	Configuración del usuario Samba	48
6.6	Configuración de <code>/etc/samba/smb.conf</code>	48
6.6.1	Archivo de configuración	48
6.6.2	Notas sobre esta configuración	49
6.7	Validación de la configuración	49
6.8	Apertura del servicio en el firewall	49
6.9	Activación de los servicios	50
6.9.1	Verificación de estado	50
6.10	Prueba local en el servidor	50
6.11	Conexión desde macOS	50
7	Monitoreo y Observabilidad	51
7.1	Contexto y objetivo	51
7.2	Node Exporter (métricas del host) en <code>:9100</code>	51
7.2.1	Crear usuario y directorios	51
7.2.2	Descargar e instalar el binario	51
7.2.3	Crear servicio <code>systemd</code>	52
7.2.4	Prueba local (métricas):	52
7.3	Prometheus (TSDB + UI) en <code>:9095</code>	53
7.3.1	Crear usuario y directorios	53

7.3.2	Descargar e instalar Prometheus	53
7.3.3	Configuración <code>prometheus.yml</code>	54
7.3.4	Servicio <code>systemd</code> (Prometheus en 9095)	54
7.3.5	Prueba local	55
7.4	Grafana (dashboards)	55
7.4.1	Agregar repositorio e instalar (repo oficial RPM)	55
7.4.2	Conectar Grafana a Prometheus	56
7.4.3	Dashboards y paneles implementados	57
7.4.4	Firewall (abre sólo lo necesario)	60
8	Instalación de Slurm para una sola máquina	61
8.1	Configuración de un nodo local con partición <code>ripley</code> , MUNGE y registros en <code>/srv/bishop</code>	61
8.2	Objetivo	61
8.3	Esquema general	62
8.4	Información del servidor	62
8.5	Preparar repositorios y dependencias	63
8.6	Construcción de los RPMs de Slurm	63
8.7	Instalación de Slurm	64
8.8	Crear usuario y grupo de Slurm	64
8.9	Configurar MUNGE	64
8.10	Crear directorios de Slurm en <code>/srv/bishop</code>	65
8.11	Detectar recursos del servidor	65
8.12	Crear <code>slurm.conf</code>	65
8.13	Crear <code>cgroup.conf</code>	66
8.14	Crear archivos de log	67
8.15	Iniciar servicios	67
8.16	Verificar instalación	67
8.17	Probar ejecución con <code>srun</code>	68
8.18	Probar ejecución con <code>sbatch</code>	68
8.19	Probar encolamiento de trabajos	69
8.20	Revisar registros de trabajos completados	70
8.21	Notas sobre <code>sacct</code>	70
8.22	Comandos útiles para usuarios	71
8.23	Comandos útiles para administración	71
8.24	Conclusión	72

Preface

El servidor **MAGINOT** es un equipo de cómputo orientado principalmente a flujos de trabajo bioinformáticos, diseñado con énfasis en **reproducibilidad, seguridad y mantenibilidad**. Su objetivo es centralizar recursos (**CPU, GPU, almacenamiento y software**) y ofrecer un esquema de **acceso remoto controlado** para usuarios autorizados.

Este documento funciona como una **bitácora técnica**: registra decisiones de configuración, comandos, rutas, servicios y cambios relevantes realizados en MAGINOT. La intención es que sirva como **evidencia** (anexo) y como **guía operativa** para mantenimiento, respaldo y futuras ampliaciones, facilitando la continuidad del servicio y la trazabilidad de las configuraciones.

Alcance de la bitácora

Incluye:

- Configuración de red y esquema de acceso remoto seguro.
- Montaje y organización de almacenamiento.
- Gestión de usuarios y políticas básicas.
- Organización de software (módulos/ambientes/contenedores) cuando aplique.
- Monitoreo y verificación de servicios.
- Registro de cambios y notas operativas.

Ficha técnica del servidor MAGINOT

- **Nombre del host:** maginot
- **Sistema operativo:** Rocky Linux 10.1 (Red Quartz)
- **Kernel:** Linux 6.12.0-124.21.1.el10_1.x86_64
- **Arquitectura:** x86_64
- **Plataforma/board:** MACHINIST E5-D8-MAX (Firmware 5.11, 2023-04-19)

CPU

- **Modelo:** Intel® Xeon® E5-2695 v4 @ 2.10 GHz
- **Sockets:** 2
- **Cores por socket:** 18
- **Hilos por core:** 2
- **Total:** 36 cores / 72 hilos
- **Frecuencia máx.:** 3.30 GHz (según `lscpu`)

Memoria RAM

- **Total usable:** ~251 GiB (según `free -h`)
- **Configuración física:** $8 \times 32 \text{ GB} = 256 \text{ GB}$
- **Tipo:** DDR4 ECC (ancho 72 bits)
- **Velocidad configurada:** 2400 MT/s

Nota: la diferencia entre 256 GB físicos y ~251 GiB visibles es normal por conversión GB/GiB y reserva del sistema.

GPU

- **Modelo:** NVIDIA GeForce RTX 3070 (8 GB VRAM)
- **Driver:** 590.48.01
- **CUDA:** 13.1 (según `nvidia-smi`)

Almacenamiento y puntos de montaje (XFS)

Disco NVMe (sistema):

- **NVMe:** Predator SSD GM7 M.2 2TB (1.9T)
 - / (root): 70G (LVM)
 - /home: 1.8T (LVM)
 - /boot y /boot/efi

Discos SATA (datos en /srv):

- /srv/call (CALL) — 4 TB (WDC WD40EZAX)
- /srv/ash (ASH) — 4 TB (WDC WD40EZRZ)
- /srv/david (DAVID) — 6 TB (WDC WD60EFRX)
- /srv/walter(WALTER) — 6 TB (WDC WD60EFRX)
- /srv/bishop(BISHOP) — 6 TB (WDC WD60PURX)

Nota: Todos los volúmenes de datos están formateados en **XFS** y montados de forma persistente bajo /srv.

Red (resumen)

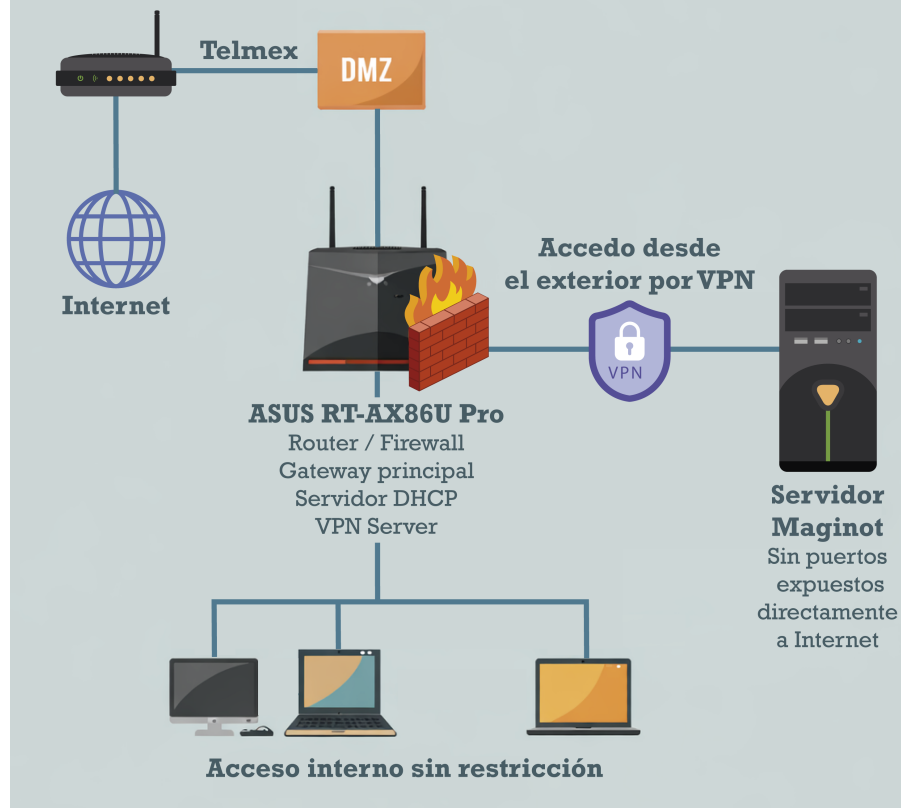
- **Interfaz LAN:** enp6s0
- **IP LAN:** 192.168.50.211/24
- **Gateway:** 192.168.50.1
- **VPN WireGuard:** wg0 → 10.8.0.1/24
- **Overlay adicional:** tailscale0 (100.105.117.111/32)
- **DNS (según resolv.conf):** 100.100.100.100 (Tailscale)

Arquitectura de red y acceso

MAGINOT se integra a la red mediante un esquema perimetral donde el enlace de Internet de **Telmex** delega conectividad hacia una **DMZ**, y el control de seguridad queda a cargo del **ASUS RT-AX86U Pro**, que opera como **gateway/firewall** y servidor de servicios de red (DHCP). En este diseño:

- El equipo Telmex entrega conectividad hacia el ASUS mediante **DMZ**.
- El ASUS funciona como **gateway principal** y **firewall** (NAT/políticas).
- El **servicio de VPN (WireGuard)** está **hospedado en MAGINOT**; el ASUS **únicamente expone/redirige** el puerto de la VPN hacia el servidor.
- **Acceso interno (LAN):** sin restricciones adicionales.
- El servidor MAGINOT **no expone puertos directamente a Internet**; el único punto de entrada remoto es el puerto de VPN publicado por el gateway.

Arquitectura de red y esquema de acceso seguro del servidor MAGINOT



1 Introducción

La **Unidad de Síntesis en Sistemática y Evolución (UniSSE)** del Instituto de Biología (IB-UNAM) se concibe como un espacio especializado para **plantear, desarrollar, fomentar y brindar apoyo** a investigaciones orientadas a entender los **procesos evolutivos** que **originan, mantienen y estructuran** la biota en distintas dimensiones (p. ej., riqueza de especies, diversidad genética, distribución geográfica) y a diferentes escalas (p. ej., macroevolución, biogeografía). Estas investigaciones se sostienen en la **síntesis de datos** provenientes de colecciones biológicas, inventarios, bases de datos cibernéticas y datos aportados por especialistas, integrados mediante enfoques analíticos y transversales dentro del IB.

En este marco, mi papel como **técnico académico responsable de los análisis** en la UniSSE se centra en **habilitar y mantener la infraestructura informática** que permite gestionar datos genómicos, implementar y optimizar software de bioinformática y filogenética comparativa, y acompañar a la comunidad académica y estudiantil del IB en la ejecución reproducible de análisis. Esta función requiere plataformas estables, seguras y escalables, capaces de sostener flujos de trabajo exigentes y diversos, con demandas crecientes de procesamiento, almacenamiento y trazabilidad.

Por ello, el **servidor “Maginot”** se plantea como una pieza de infraestructura estratégica para la UniSSE: un sistema GNU/Linux diseñado para ofrecer **servicio multiusuario confiable**, facilitar la **reproducibilidad**, asegurar el **uso ordenado y justo de recursos**, y permitir **observabilidad** y mantenimiento continuo para garantizar la continuidad operativa de los análisis en los que apoya la Unidad.

1.1 Objetivo principal

Diseñar e implementar un servidor GNU/Linux multiusuario, seguro y observable que permita la ejecución rápida y reproducible de pipelines bioinformáticos mediante contenedores y control de recursos, con almacenamiento confiable y cuotas, para dar un servicio estable.

1.2 Objetivos específicos

- **Multiusuario y control de acceso:** cuentas, grupos, permisos, políticas de autenticación y buenas prácticas de uso.

- **Ejecución reproducible:** estandarizar la ejecución de pipelines con contenedores (p. ej., Apptainer/Podman) para minimizar variaciones por dependencias.
- **Control y asignación de recursos:** lineamientos y herramientas para limitar/administrar CPU, RAM y tiempos de ejecución (p. ej., colas o políticas de prioridad según el diseño del servicio).
- **Almacenamiento confiable:** estructura de directorios de trabajo, respaldos, y **cuotas** por usuario/proyecto para sostener operación a largo plazo.
- **Seguridad operativa:** endurecimiento básico del sistema, registro de eventos y medidas contra accesos no autorizados.
- **Observabilidad y mantenimiento:** monitoreo de rendimiento/uso (CPU, RAM, disco, red), alertas y bitácoras para diagnóstico y continuidad del servicio.
- **Documentación y soporte:** guías mínimas para usuarios (acceso, ejecución, buenas prácticas) y registro técnico para mantenimiento y auditoría.

2 Validación y configuración de discos

2.1 Contexto y objetivo

Este servidor integra discos de almacenamiento adicionales como **volúmenes independientes (sin RAID)** para ampliar la capacidad y alojar archivos **no críticos** (material temporal, descargas, datos replicables, etc.). A diferencia de un volumen redundante, estos discos **no cuentan con tolerancia a fallas**, por lo que su uso se limita a información prescindible o respaldada en otro sitio.

2.1.1 Aclaración sobre el cambio de estrategia (RAID1 → volúmenes individuales)

En la planeación inicial se consideró implementar **RAID1 (espejo)** para contar con redundancia. Sin embargo, durante la implementación se decidió **no continuar con RAID1**, ya que este esquema reduce la **capacidad útil al 50%** (duplica la información en ambos discos).

Dado que en esta etapa el objetivo operativo es **maximizar el espacio disponible**, se optó por integrar los discos como **volúmenes individuales**. En consecuencia, estos volúmenes se destinan a **datos no críticos, temporales o replicables**, o a información que cuenta con **respaldo externo**.

Nota: En esta estrategia, la protección se gestiona mediante **copias de seguridad y/o replicación externa**, no mediante redundancia local.

2.2 Inventario y detección de dispositivos

Para identificar discos, particiones y sistemas de archivos:

```
lsblk -o NAME,SIZE,FSTYPE,LABEL,UUID,MOUNTPOINTS
lsblk -d -o NAME,SIZE,MODEL,SERIAL,WWN,TRAN,ROTA
```

```
stoich@maginot:~$ lsblk -o NAME,SIZE,FSTYPE,LABEL,UUID,MOUNTPOINTS
NAME        SIZE FSTYPE LABEL  UUID                                MOUNTPOINTS
sda          3.6T
├─sda1       3.6T xfs    CALL   e8a3ab92-deeb-4754-bb60-5de622f8e3a6 /srv/call
├─sdb        5.5T
├─sdb1       5.5T xfs    DAVID  882a9ab2-205a-4bc6-8712-bb270620b942 /srv/david
├─sdc        5.5T
├─sdc1       5.5T xfs    WALTER 2dc26afe-8ce7-49a9-9dcb-4d585feba4d2 /srv/walter
├─sdd        5.5T
├─sdd1       5.5T xfs    BISHOP 4389b7e6-5a49-4be8-addd-18a91b325d15 /srv/bishop
├─sde        3.6T
├─sde1       3.6T xfs    ASH     73d57dda-cd7c-4070-8cb7-126792451a2c /srv/ash
└─nvme0n1    1.9T
   └─nvme0n1p1 600M vfat    C188-8210 /boot/efi
      └─nvme0n1p2 1G xfs     bee7fc27-6919-4a15-8762-d51014d25f85 /boot
         └─nvme0n1p3 1.9T LVM2_member YacppU-dbAI-6HdQ-0gz4-01XB-07zU-zGGRvs
            └─rl_stoich-root 70G xfs     b07b2a24-e076-4da6-9f11-ccb3d052f1b0 /
               └─rl_stoich-swap 32G swap   83c40646-e5f8-4b5b-af2d-cec20a17acf1 [SWAP]
                  └─rl_stoich-home 1.8T xfs     2271ffd0-4323-4bba-91ed-0dd68d68e4d5 /home

stoich@maginot:~$ lsblk -d -o NAME,SIZE,MODEL,SERIAL,WWN,TRAN,ROTA
NAME        SIZE MODEL          SERIAL          WWN                                TRAN  ROTA
sda          3.6T WDC WD40EZAX-00C8UB0 WD-WX22D63R0NEE 0x50014ee215eb251d sata  1
sdb          5.5T WDC WD60EFRX-68MYMN1 WD-WX21D1400980 0x50014ee20be941ee sata  1
sdc          5.5T WDC WD60EFRX-68MYMN1 WD-WX11D153XZ82 0x50014ee20be17e64 sata  1
sdd          5.5T WDC WD60PURX-64T0ZY0 WD-WX11D46CH2R4 0x50014ee262f6f88a sata  1
sde          3.6T WDC WD40EZRX-00GXC00 WD-WCC7K7HCK13V 0x50014ee265becf3a sata  1
nvme0n1     1.9T Predator SSD GM7 M.2 2TB PSBH53430100218 nvme.1e4b-505342483533343330313030323138-5072656461746f722053534420474d37204d2e3220325442-00000001 nvme  0
stoich@maginot:~$
```

2.2.1 Sobre los nombres /dev/sdX

Los nombres sda, sdb, sdc, etc. **pueden cambiar entre reinicios** por el orden de detección del kernel (puertos SATA, tiempos de inicialización, etc.). Para **evitar confusiones al seleccionar discos**, se recomienda usar rutas estables como:

- /dev/disk/by-id/... (muy estable, incluye modelo/serial)
- UUID= o LABEL= para montaje en /etc/fstab (estable para montar)

Ejemplo:

```
ls -l /dev/disk/by-id/ | head
readlink -f /dev/disk/by-id/ata-XXXX
```

2.3 Estado actual (volúmenes individuales integrados)

Los discos de datos quedaron integrados como XFS y montados bajo /srv/*:

- **BISHOP** → /srv/bishop
- **DAVID** → /srv/david
- **WALTER** → /srv/walter
- **ASH** → /srv/ash
- **CALL** → /srv/call

Comprobación rápida:

```
df -hT /srv/bishop /srv/david /srv/walter /srv/ash /srv/call
```

```
stoich@maginot:~/Documents/maginot_project/images/disks$ df -hT /srv/bishop /srv/david /srv/walter /srv/ash /srv/call
Filesystem      Type  Size  Used Avail Use% Mounted on
/dev/sdd1       xfs   5.5T  1.4T  4.2T  25% /srv/bishop
/dev/sdb1       xfs   5.5T  2.6T  3.0T  47% /srv/david
/dev/sdc1       xfs   5.5T  1.4T  4.1T  26% /srv/walter
/dev/sde1       xfs   3.7T  402G  3.3T  11% /srv/ash
/dev/sda1       xfs   3.7T  240G  3.5T   7% /srv/call
stoich@maginot:~/Documents/maginot_project/images/disks$
```

2.4 Procedimiento estándar (molde) para agregar discos nuevos sin RAID

Este procedimiento es destructivo: borra particiones y firmas del disco seleccionado.

2.4.1 Identificar el disco por MODEL/SERIAL y/o by-id

```
lsblk -d -o NAME,SIZE,MODEL,SERIAL
ls -l /dev/disk/by-id/ | egrep 'WDC|ata-' | head
```

Definir una variable (recomendado) para evitar equivocaciones:

```
DISK="/dev/disk/by-id/ata-XXXXXXXXXXXX" # <- reemplaza por el correcto
readlink -f "$DISK"
```

2.4.2 Desmontar si estaba montado

```
sudo umount -R /srv/<montaje> 2>/dev/null || true
```

2.4.3 Borrar firmas y tabla de particiones

```
sudo wipefs -a "$DISK"
sudo sgdisk --zap-all "$DISK"
sudo partprobe "$DISK" || true
```

2.4.4 Crear GPT + 1 partición (100%)

```
sudo parted -s "$DISK" mklabel gpt
sudo parted -s "$DISK" mkpart primary xfs 1MiB 100%
sudo partprobe "$DISK"
```

Detectar la partición creada:

```
lsblk "$DISK"
```

Si usas by-id, la partición suele verse como ...-part1:

```
PART="${DISK}-part1"
readlink -f "$PART"
```

2.4.5 Formatear a XFS con etiqueta

```
LABEL="NOMBRE"      # ej. BISHOP / DAVID / WALTER / ASH / CALL
sudo mkfs.xfs -f -L "$LABEL" "$PART"
sudo blkid "$PART"
```

2.4.6 Crear punto de montaje y montar

```
MP="/srv/${LABEL,,}"    # convierte a minúsculas (bash)
sudo mkdir -p "$MP"
sudo mount "$PART" "$MP"
findmnt "$MP"
```

2.4.7 Agregar a /etc/fstab y probar

```
grep -n " $MP " /etc/fstab || true
echo "LABEL=$LABEL $MP xfs defaults,noatime,nofail,x-systemd.device-timeout=10 0
    0" | sudo tee -a /etc/fstab

sudo systemctl daemon-reload
sudo umount "$MP"
sudo mount -a
findmnt "$MP"
df -hT "$MP"
```

2.4.8 Verificación final (rápida)

```
lsblk -o NAME,SIZE,FSTYPE,LABEL,UUID,MOUNTPOINTS
sudo blkid | egrep 'BISHOP|DAVID|WALTER|ASH|CALL' || true
```

```
stoich@maginot:~$ lsblk -o NAME,SIZE,FSTYPE,LABEL,UUID,MOUNTPOINTS
NAME                                SIZE FSTYPE LABEL UUID                                MOUNTPOINTS
├─sda                                3.6T
├─┬sda1                             3.6T xfs    CALL  e8a3ab92-deeb-4754-bb60-5de622f8e3a6  /srv/call
├─sdb                                5.5T
├─┬sdb1                             5.5T xfs    DAVID 882a9ab2-205a-4bc6-8712-bb270620b942  /srv/david
├─sdc                                3.6T
├─┬sdc1                             3.6T xfs    ASH    73d57dda-cd7c-4070-8cb7-126792451a2c  /srv/ash
├─sdd                                5.5T
├─┬sdd1                             5.5T xfs    WALTER 2dc26afe-8ce7-49a9-9dcb-4d585feba4d2  /srv/walter
├─sde                                5.5T
├─┬sde1                             5.5T xfs    BISHOP 4389b7e6-5a49-4be8-addd-18a91b325d15  /srv/bishop
├─nvme0n1                             1.9T
├─┬nvme0n1p1                       600M vfat    C188-8210                               /boot/efi
├─┬nvme0n1p2                       1G xfs      bee7fc27-6919-4a15-8762-d51014d25f85  /boot
├─┬nvme0n1p3                       1.9T LVM2_member YacppU-dbAI-6HdQ-Ogz4-01XB-07zU-zGGRvs
├─┬┬rl_stoich-root                 70G xfs      b07b2a24-e076-4da6-9f11-ccb3d052f1b0  /
├─┬┬rl_stoich-swap                 32G swap     83c40646-e5f8-4b5b-af2d-cec20a17acf1  [SWAP]
├─┬┬rl_stoich-home                 1.8T xfs      2271ffd0-4323-4bba-91ed-0dd68d68e4d5  /home
└─sdd1                             5.5T
/dev/sdd1: LABEL="WALTER" UUID="2dc26afe-8ce7-49a9-9dcb-4d585feba4d2" BLOCK_SIZE="4096" TYPE="xfs" PARTLABEL="primary" PARTUUID="3cea8f74-ab30-4b17-80b2-d30c2ae4d474"
/dev/sdb1: LABEL="DAVID" UUID="882a9ab2-205a-4bc6-8712-bb270620b942" BLOCK_SIZE="4096" TYPE="xfs" PARTLABEL="primary" PARTUUID="b3008c65-41a3-4200-855c-5ec7bf1d037d"
/dev/sde1: LABEL="BISHOP" UUID="4389b7e6-5a49-4be8-addd-18a91b325d15" BLOCK_SIZE="4096" TYPE="xfs" PARTLABEL="primary" PARTUUID="7de24d9c-8407-48f4-bb4e-b2bf41ac6f64"
/dev/sdc1: LABEL="ASH" UUID="73d57dda-cd7c-4070-8cb7-126792451a2c" BLOCK_SIZE="4096" TYPE="xfs" PARTLABEL="primary" PARTUUID="d32c88a1-1af9-4b9c-a1c7-df15b0c58a1f"
/dev/sda1: LABEL="CALL" UUID="e8a3ab92-deeb-4754-bb60-5de622f8e3a6" BLOCK_SIZE="4096" TYPE="xfs" PARTLABEL="primary" PARTUUID="8907775c-d2d8-4b81-bfcb-f24aef89d97b"
```


3 VPN con WireGuard

Contexto y objetivo

Por razones de seguridad, el servidor **MAGINOT** no debe ser accesible directamente desde Internet (por ejemplo, evitando publicar SSH, paneles web o puertos de servicios hacia el exterior). En su lugar, el acceso remoto se realiza **exclusivamente a través de una VPN**, de modo que solo equipos autenticados puedan entrar a la red privada del servidor y consumir servicios internos como si estuvieran en la red local.

Por ello, se implementó **WireGuard** como VPN en Rocky Linux, configurado para escuchar en el puerto **13420/UDP**, con el fin de que toda administración y acceso remoto a MAGINOT (SSH, RStudio, Grafana, etc.) ocurra únicamente a través del túnel VPN, reduciendo la exposición pública de servicios y centralizando el control de acceso.

3.1 Instalación de WireGuard

```
sudo dnf -y update
sudo dnf -y install epel-release
sudo dnf -y install wireguard-tools
```

3.2 Cargar el módulo del kernel y verificar

```
sudo modprobe wireguard
lsmod | grep wireguard || true
```

```
stoich@maginot:~/Documents/maginot_project$ sudo modprobe wireguard
stoich@maginot:~/Documents/maginot_project$ lsmod | grep wireguard || true
wireguard                118784  0
ip6_udp_tunnel           16384  1 wireguard
udp_tunnel               36864  1 wireguard
curve25519_x86_64        36864  1 wireguard
libcurve25519_generic     49152  2 curve25519_x86_64,wireguard
stoich@maginot:~/Documents/maginot_project$
```

La presencia del módulo `wireguard` en `lsmod` confirma que el kernel puede crear interfaces WireGuard (por ejemplo, `wg0`). Los módulos `udp_tunnel` e `ip6_udp_tunnel` son dependencias relacionadas con encapsulamiento UDP.

3.3 Identificar la interfaz de salida (NAT)

WireGuard requiere conocer la interfaz por la que sale el tráfico para configurar NAT.

```
IFACE=$(ip route get 1.1.1.1 | awk '{print $5; exit}')
echo "Interfaz de salida: $IFACE"
```

Salida esperada:

```
Interfaz de salida: enp6s0
```

3.4 Crear llaves del servidor

Cambiar a modo root

```
mkdir -p /etc/wireguard
cd /etc/wireguard
umask 077

wg genkey | sudo tee server.key >/dev/null
cat server.key | sudo wg pubkey | sudo tee server.pub >/dev/null

echo "Server public key:"
sudo cat /etc/wireguard/server.pub
```

La **public key** del servidor (`server.pub`) se utiliza para configurar clientes.

3.5 Crear la interfaz `wg0` (configuración del servidor)

Se utilizará la red VPN `10.8.0.0/24`:

- Servidor VPN: `10.8.0.1`
- Puerto: `13420/UDP`
- NAT saliendo por: `enp6s0`

Crear /etc/wireguard/wg0.conf:

```
SERVER_PRIV=$(sudo cat /etc/wireguard/server.key)

sudo tee /etc/wireguard/wg0.conf >/dev/null <<EOF
[Interface]
Address = 10.8.0.1/24
ListenPort = 13420
PrivateKey = ${SERVER_PRIV}

# NAT + forwarding
PostUp = iptables -A FORWARD -i %i -j ACCEPT; iptables -A FORWARD -o %i -j ACCEPT;
        iptables -t nat -A POSTROUTING -o enp6s0 -j MASQUERADE
PostDown = iptables -D FORWARD -i %i -j ACCEPT; iptables -D FORWARD -o %i -j ACCEPT;
        iptables -t nat -D POSTROUTING -o enp6s0 -j MASQUERADE
EOF

sudo chmod 600 /etc/wireguard/wg0.conf /etc/wireguard/server.key
```

3.6 Habilitar IP forwarding

```
echo 'net.ipv4.ip_forward = 1' | sudo tee /etc/sysctl.d/99-wireguard.conf
sysctl --system | grep -i ip_forward || true
```

Salida esperada :

```
net.ipv4.ip_forward = 1
```

3.7 Abrir el puerto 13420/udp en firewalld

```
firewall-cmd --permanent --add-port=13420/udp
firewall-cmd --reload
firewall-cmd --list-ports | tr ' ' '\n' | grep 13420 || true
```

3.8 Activar WireGuard (wg-quick@wg0)

```
sudo systemctl enable --now wg-quick@wg0
sudo systemctl status wg-quick@wg0 --no-pager -l
```

```

root@maginot:/etc/wireguard# sudo systemctl status wg-quick@wg0 --no-pager -l
● wg-quick@wg0.service - WireGuard via wg-quick(8) for wg0
   Loaded: loaded (/usr/lib/systemd/system/wg-quick@.service; enabled; preset: disabled)
   Active: active (exited) since Mon 2026-01-05 10:49:06 CST; 4s ago
  Invocation: c2499943c6a24a5e9c255af8a66537bf
     Docs: man:wg-quick(8)
           man:wg(8)
           https://www.wireguard.com/
           https://www.wireguard.com/quickstart/
           https://git.zx2c4.com/wireguard-tools/about/src/man/wg-quick.8
           https://git.zx2c4.com/wireguard-tools/about/src/man/wg.8
   Process: 10924 ExecStart=/usr/bin/wg-quick up wg0 (code=exited, status=0/SUCCESS)
  Main PID: 10924 (code=exited, status=0/SUCCESS)
  Mem peak: 4.7M
    CPU: 55ms

Jan 05 10:49:06 maginot systemd[1]: Starting wg-quick@wg0.service - WireGuard via wg-quick(8) for wg0...
Jan 05 10:49:06 maginot wg-quick[10924]: [#] ip link add wg0 type wireguard
Jan 05 10:49:06 maginot wg-quick[10924]: [#] wg setconf wg0 /dev/fd/63
Jan 05 10:49:06 maginot wg-quick[10924]: [#] ip -4 address add 10.8.0.1/24 dev wg0
Jan 05 10:49:06 maginot wg-quick[10924]: [#] ip link set mtu 1420 up dev wg0
Jan 05 10:49:06 maginot wg-quick[10924]: [#] iptables -A FORWARD -i wg0 -j ACCEPT; iptables -A FORWARD -o wg0 -j ACCEPT; iptables -t nat -A POSTROUTING -o enp6s0 -j MASQUERADE
Jan 05 10:49:06 maginot systemd[1]: Finished wg-quick@wg0.service - WireGuard via wg-quick(8) for wg0.
root@maginot:/etc/wireguard#

```

3.9 Estado de WireGuard

```

wg show
ss -lunp | grep 13420 || true
ip -br a | grep wg0 || true

```

Resultado: WireGuard quedó habilitado en MAGINOT: interfaz wg0 activa, IP asignada y puerto **13420/UDP** en escucha.

3.10 Crear clientes por nombre

3.10.1 Script de alta: wg-add-client (crea peer + archivo NOMBRE.conf)

Este script:

1. asigna automáticamente la siguiente IP libre en 10.8.0.0/24,
2. genera llaves del cliente,
3. crea el archivo NOMBRE.conf (full-tunnel),
4. agrega el peer a wg0.conf con comentario,
5. reinicia wg-quick@wg0.

3.10.2 Instalar el script

```
sudo tee /usr/local/sbin/wg-add-client >/dev/null <<'EOF'
#!/usr/bin/env bash
set -euo pipefail

NAME="${1:-}"
ENDPOINT_HOST="${2:-}"
PORT="${3:-13420}"

if [[ -z "$NAME" || -z "$ENDPOINT_HOST" ]]; then
    echo "Uso: sudo wg-add-client NOMBRE ENDPOINT_HOST [PUERTO]"
    echo "Ejemplo: sudo wg-add-client karen stoichsse.ddns.net 13420"
    exit 1
fi

NAME="$(echo "$NAME" | tr ' ' '_' | tr -cd '[:alnum:]_-' )"

WGDIR="/etc/wireguard"
WGIF="wg0"
NET="10.8.0"
CLIENT_DIR="${WGDIR}/clients"

# checks mínimos
[[ -r "${WGDIR}/server.pub" ]] || { echo "ERROR: falta ${WGDIR}/server.pub"; exit 1; }
[[ -r "${WGDIR}/${WGIF}.conf" ]] || { echo "ERROR: falta ${WGDIR}/${WGIF}.conf"; exit 1; }

mkdir -p "$CLIENT_DIR"
chmod 700 "$CLIENT_DIR"

cd "$WGDIR"
umask 077

# Encontrar IPs ya usadas (si no hay matches, no fallar)
USED=$(
    { grep -oE 'AllowedIPs = 10\.8\.0\.[0-9]+/32' "${WGDIR}/${WGIF}.conf" 2>/dev/null
    || true; } \
    | awk -F'[/.]' '{print $4}' | sort -n | uniq
)

IP=2
while echo "$USED" | grep -qx "$IP"; do IP=$((IP+1)); done
[[ $IP -le 254 ]] || { echo "ERROR: No hay IPs libres en ${NET}.0/24"; exit 1; }

# Evitar duplicados por nombre
if [[ -e "${CLIENT_DIR}/${NAME}.conf" ]]; then
```

```

    echo "ERROR: ya existe ${CLIENT_DIR}/${NAME}.conf"
    exit 1
fi

# Generar llaves del cliente
wg genkey | tee "${CLIENT_DIR}/${NAME}.key" >/dev/null
cat "${CLIENT_DIR}/${NAME}.key" | wg pubkey > "${CLIENT_DIR}/${NAME}.pub"

# Crear config full-tunnel (Internet por VPN)
cat > "${CLIENT_DIR}/${NAME}.conf" <<CFG
[Interface]
PrivateKey = $(cat "${CLIENT_DIR}/${NAME}.key")
Address = ${NET}.${IP}/24
DNS = 1.1.1.1

[Peer]
PublicKey = $(cat "${WGDIR}/server.pub")
Endpoint = ${ENDPOINT_HOST}:${PORT}
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 25
CFG

chmod 600 "${CLIENT_DIR}/${NAME}.conf" "${CLIENT_DIR}/${NAME}.key"
        "${CLIENT_DIR}/${NAME}.pub"

# Agregar peer al servidor (persistente)
cat >> "${WGDIR}/${WGIF}.conf" <<PEER

# ${NAME}
[Peer]
PublicKey = $(cat "${CLIENT_DIR}/${NAME}.pub")
AllowedIPs = ${NET}.${IP}/32
PEER

systemctl restart "wg-quick@${WGIF}"

echo "OK: ${CLIENT_DIR}/${NAME}.conf"
echo "IP asignada: ${NET}.${IP}"
echo "PublicKey cliente: $(cat "${CLIENT_DIR}/${NAME}.pub")"
EOF

sudo chmod +x /usr/local/sbin/wg-add-client

```

3.10.3 Crear un cliente (por nombre)

Ejemplo:

```
sudo /usr/local/sbin/wg-add-client stoich stoichsse.ddns.net 13420
```

```
stoich@maginot:~/Documents/maginot_project$ sudo /usr/local/sbin/wg-add-client stoich stoichsse.ddns.net 13420
OK: /etc/wireguard/clients/stoich.conf
IP asignada: 10.8.0.2
PublicKey cliente: 6VnSSdR3YPYfwg9BLaA0sbwLRgMawAs2y3b5+zP31Hw=
```

El archivo quedará en:

```
/etc/wireguard/clients/stoich.conf
```

Listar clientes:

```
ls -lh /etc/wireguard/clients/
```

Confirmar que el peer quedó cargado:

```
sudo wg show
```

3.11 Copiar el .conf a un home (para transferencia a teléfono/PC)

```
sudo cp /etc/wireguard/clients/stoich.conf /home/stoich/
sudo chown stoich:stoich /home/stoich/stoich.conf
chmod 600 /home/stoich/stoich.conf
ls -lh /home/stoich/stoich.conf
```

3.12 Revocar un cliente (baja)

1. Eliminar archivos del cliente:

```
sudo rm -f /etc/wireguard/clients/stoich.{conf,key,pub}
```

2. Eliminar el bloque del peer en el servidor:

- Buscar # stoich en /etc/wireguard/wg0.conf y borrar ese bloque completo [Peer].

3. Reiniciar WireGuard y verificar:

```
sudo systemctl restart wg-quick@wg0
sudo wg show
```

4 Fortalecimiento de Acceso y Autenticación

4.1 Contraseñas

4.1.1 Contexto y objetivo

MAGINOT es un servidor multiusuario orientado a análisis bioinformáticos y administración de datos de investigación. En este contexto, el acceso a cuentas es un punto crítico de seguridad: una contraseña débil o compartida puede comprometer no solo la cuenta individual, sino también recursos, datos y servicios del sistema.

Por ello, se implementó una **política mínima de contraseñas** con dos principios:

1. **Responsabilidad individual:** cada usuario es responsable de la seguridad de su contraseña.

- No está permitido compartir contraseñas (ni con colegas, estudiantes o terceros).
- Toda actividad realizada desde una cuenta se atribuye al titular de esa cuenta.

2. **Arranque seguro de cuentas nuevas:** el administrador asigna una **contraseña temporal inicial** durante la creación de la cuenta.

- En el ****primer inicio de sesión****, la contraseña temporal debe cambiarse por una nueva, personal e intransferible.
- El cambio inicial se acompaña de un mensaje con requisitos mínimos para evitar contraseñas triviales.

4.1.2 Requisitos mínimos de contraseña

- Longitud mínima: **10** caracteres
- Debe incluir al menos:

- ****1 minúscula****
- ****1 mayúscula****
- ****1 número****
- ****1 símbolo**** (carácter especial)

4.2 Definir la política en `/etc/security/pwquality.conf`

Este archivo aplica y audita la política de complejidad a nivel sistema; lo utilizan `passwd` y `pam_pwquality.so`.

Editar el archivo:

```
sudo nano /etc/security/pwquality.conf
```

Ajustar (o asegurar) los siguientes parámetros:

```
# Longitud mínima de la contraseña
minlen = 10

# Mínimo 1 letra minúscula
lcredit = -1

# Mínimo 1 letra mayúscula
ucredit = -1

# Mínimo 1 dígito
dcredit = -1

# Mínimo 1 símbolo
ocredit = -1
```

4.2.1 Verificación inmediata (para confirmar que ya aplica)

Para validar que la política se encuentra activa, se ejecutó:

```
passwd usuario
```

Se probó una contraseña que no cumpliera (por ejemplo, sin símbolo o con menos de 10 caracteres) y el sistema la rechazó. Posteriormente, se probó una contraseña que sí cumpliera, verificando que el cambio se completara correctamente.

4.2.2 Crear el script /usr/local/sbin/maginot-add-user

Con el fin de estandarizar el alta de cuentas, se creó un script administrativo para:

- crear usuario y directorio home,
- asignar una contraseña temporal robusta,
- forzar cambio en el primer inicio de sesión,
- (opcional) asignar grupos.

Crear el archivo:

```
sudo tee /usr/local/sbin/maginot-add-user >/dev/null <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
export LC_ALL=C

usage() {
    cat <<'USAGE'
Uso:
    sudo maginot-add-user [opciones] usuario

Opciones:
    -g Grupos separados por coma (ej: mamba,docker)
    -s Shell (default: /bin/bash)
    -c Comentario/GECOS (nombre completo)
    -b Directorio base para homes (default: /home) -> home será: BASE/usuario
    -d Home directory explícito (sobrescribe -b), ej: /mnt/fury/home/usuario
    -l Longitud de contraseña temporal (default: 18; mínimo efectivo 10)
USAGE
}

GROUPS=""
SHELL="/bin/bash"
COMMENT=""
HOME_BASE="/home"
HOME_DIR=""
PASS_LEN="18"

while getopts ":g:s:c:b:d:l:h" opt; do
    case "$opt" in
        g) GROUPS="$OPTARG" ;;
        s) SHELL="$OPTARG" ;;
        c) COMMENT="$OPTARG" ;;
        b) HOME_BASE="$OPTARG" ;;
        d) HOME_DIR="$OPTARG" ;;
        l) PASS_LEN="$OPTARG" ;;
```

```

    h) usage; exit 0 ;;
    \?) echo "Opción inválida: -$OPTARG" >&2; usage; exit 2 ;;
    :) echo "Falta argumento para -$OPTARG" >&2; usage; exit 2 ;;
esac
done
shift $((OPTIND - 1))

USER="${1:-}"
if [[ -z "${USER}" ]]; then
    echo "Error: Debes indicar un usuario." >&2
    usage
    exit 2
fi

if [[ "$(id -u)" -ne 0 ]]; then
    echo "Error: Ejecuta como root (usa sudo)." >&2
    exit 1
fi

# Normaliza a minúsculas (recomendado)
USER="${USER,,}"

# Validación de username (minúsculas, números, _ o -)
if ! [[ "$USER" =~ ^[a-z_][a-z0-9_-]*$ ]]; then
    echo "Error: username inválido '$USER' (usa minúsculas, números, _ o -; no
    empieces con número)." >&2
    exit 2
fi

if id "$USER" &>/dev/null; then
    echo "Error: El usuario '$USER' ya existe." >&2
    exit 1
fi

# Home dir: si no se dio -d, se construye con -b
if [[ -z "$HOME_DIR" ]]; then
    HOME_DIR="${HOME_BASE%}/$USER"
fi

# Asegura que el directorio padre exista
BASE_DIR="$(dirname "$HOME_DIR")"
if [[ ! -d "$BASE_DIR" ]]; then
    mkdir -p "$BASE_DIR"
    chmod 755 "$BASE_DIR"
fi

# --- Generador de contraseña temporal (sin pipes) ---
# Símbolos permitidos (evitamos ':' y espacios)

```

```

SYMBOLS='@#%+=_!~*?-'

if command -v python3 >/dev/null 2>&1; then
    PASS="$(PASS_LEN="$PASS_LEN" SYMBOLS="$SYMBOLS" python3 - <<'PY'
import os, secrets, string
length = int(os.environ.get("PASS_LEN", "18"))
symbols = os.environ.get("SYMBOLS", "@#%+=_!~*?-'")
alphabet = string.ascii_letters + string.digits + symbols

while True:
    pw = ''.join(secrets.choice(alphabet) for _ in range(length))
    if len(pw) >= 10 \
        and any(c.islower() for c in pw) \
        and any(c.isupper() for c in pw) \
        and any(c.isdigit() for c in pw) \
        and any(c in symbols for c in pw):
        print(pw)
        break
PY
)"
else
    echo "Error: python3 no está disponible (instálalo o ajusta el generador)." >&2
    exit 1
fi

# --- Crear usuario (con home en ruta personalizada) ---
if [[ -n "$COMMENT" ]]; then
    useradd -m -d "$HOME_DIR" -k /etc/skel -s "$SHELL" -c "$COMMENT" "$USER"
else
    useradd -m -d "$HOME_DIR" -k /etc/skel -s "$SHELL" "$USER"
fi

# Por si acaso: si no se creó home, créalo
if [[ ! -d "$HOME_DIR" ]]; then
    mkdir -p "$HOME_DIR"
    chown "$USER:$USER" "$HOME_DIR"
    chmod 700 "$HOME_DIR"
fi

# Asignar contraseña temporal
echo "${USER}:${PASS}" | chpasswd

# Forzar cambio en primer login
chage -d 0 "$USER"

# Agregar a grupos (opcional)
if [[ -n "$GROUPS" ]]; then
    usermod -aG "${GROUPS//,/ }" "$USER"

```

```
fi

echo "Usuario creado: $USER"
echo "Home: $HOME_DIR"
echo "Contraseña temporal: $PASS"
echo "Acción: Se forzó cambio de contraseña al primer login (chage -d 0)."
```

EOF

Asignar permisos correctos:

```
sudo tee /usr/bin/maginot-add-user >/dev/null <<'EOF'
#!/usr/bin/env bash
exec /usr/local/sbin/maginot-add-user "$@"
EOF

sudo chmod 750 /usr/local/sbin/maginot-add-user
sudo chown root:root /usr/local/sbin/maginot-add-user
```

Nota de seguridad: el script imprime la contraseña temporal en pantalla. Se recomienda compartirla **una sola vez** y por un canal seguro.

4.2.3 Uso del script (alta de usuarios)

Ejemplo simple:

```
sudo maginot-add-user usuario_nuevo
```

Con grupos:

```
sudo maginot-add-user -g mamba,docker usuario_nuevo
```

Con nombre completo:

```
sudo maginot-add-user -c "Juan Pérez" usuario_nuevo
```

Cómo crear el home en otra ruta:

```
sudo maginot-add-user -b /srv/fury/home usuario1
```

4.2.4 Mensajes de política (una sola vez)

Mensaje antes del login SSH (opcional): `/etc/issue.net` + Banner

```
sudo tee /etc/issue.net >/dev/null <<'EOF'
Servidor MAGINOT - Aviso de seguridad
No compartas tu password.
EOF
```

Activa Banner en SSH:

```
sudo grep -n '^Banner' /etc/ssh/sshd_config || true
```

Si no aparece, agrega:

```
echo 'Banner /etc/issue.net' | sudo tee -a /etc/ssh/sshd_config
sudo systemctl restart sshd
```

4.3 Configuración de SSH

4.3.1 Contexto y objetivo

SSH es el punto de entrada principal para administración remota. Para reducir la superficie de ataque y mitigar intentos de fuerza bruta, se endureció la configuración del demonio `sshd` con medidas básicas: deshabilitar acceso directo de `root`, mover el puerto por defecto, limitar intentos de autenticación, prohibir contraseñas vacías y reducir el tiempo permitido para completar el inicio de sesión.

4.3.2 Cambios en `/etc/ssh/sshd_config`

Respaldar el archivo:

```
sudo cp -a /etc/ssh/sshd_config /etc/ssh/sshd_config.bak.$(date +%F_%H%M)
```

Editar configuración:

```
sudo nano /etc/ssh/sshd_config
```

Asegurar estas directivas (pueden colocarse al final para que prevalezcan):

```
# --- MAGINOT: Hardening SSH ---

# Cambiar puerto (ejemplo: 14420)
Port 14420

# No permitir login directo de root
PermitRootLogin no

# No permitir contraseñas vacías
PermitEmptyPasswords no

# Limitar intentos de autenticación por conexión
MaxAuthTries 6

# Reducir el tiempo para completar el login
LoginGraceTime 2m
```

Validar sintaxis y recargar:

```
sudo sshd -t
sudo systemctl reload sshd
```

4.4 Activar Fail2Ban

Instalar:

```
sudo dnf install -y fail2ban
```

Habilitar y arrancar:

```
sudo systemctl enable --now fail2ban
sudo systemctl status fail2ban --no-pager -l
```

Crear un archivo dedicado:

```
sudo tee /etc/fail2ban/jail.d/sshd.local >/dev/null <<'EOF'
[sshd]
enabled = true

# Tu puerto SSH (ajústalo si usas otro)
port = 14420

# En Rocky, lo más robusto suele ser leer del journal
backend = systemd
```

```
# Política: 3 intentos en 10 minutos => ban 1 hora
maxretry = 3
findtime = 10m
bantime = 1h

# No te bloques a ti mismo (agrega aquí tu(s) IP(s) de confianza)
ignoreip = 127.0.0.1/8 ::1 132.248.13.8/32

# Acción usando firewalld (recomendado en Rocky)
banaction = firewallcmd-ipset
EOF
```

Recargar y verificar:

```
sudo fail2ban-client reload
sudo fail2ban-client status
sudo fail2ban-client status sshd
```

Revisar eventos:

```
sudo journalctl -u sshd -n 200 --no-pager
sudo tail -n 200 /var/log/fail2ban.log
```

Desbanear una IP (si aplica):

```
sudo fail2ban-client set sshd unbanip X.X.X.X
```


5 Contenedores y Entornos Reproducibles

5.1 Contexto y objetivo

En bioinformática es común que distintos proyectos (o incluso distintas etapas de un mismo pipeline) requieran **conjuntos de herramientas diferentes** y, con frecuencia, **versiones distintas del mismo software**. Esto puede deberse a cambios en dependencias, parámetros, formatos, actualizaciones de algoritmos o, simplemente, a la necesidad de repetir exactamente un análisis ya utilizado en una **publicación, tesis o reporte técnico**. En un servidor multiusuario, instalar software “a nivel sistema” suele ser insuficiente y puede provocar conflictos (librerías incompatibles, rutas mezcladas, ejecutables duplicados), además de aumentar el costo de mantenimiento.

Por ello, el objetivo de esta sección es **implementar y documentar** una estrategia de gestión de software que habilite la ejecución **rápida, multiversión y reproducible** de pipelines bioinformáticos, manteniendo un sistema base estable y facilitando la **trazabilidad** de versiones. Para lograrlo se combinan herramientas complementarias:

- **Contenedores** (Docker, Podman, Apptainer) para encapsular aplicaciones y dependencias en imágenes ejecutables de forma consistente, favoreciendo **portabilidad y aislamiento**.
- **Entornos con Micromamba** para crear entornos ligeros por proyecto con versiones específicas de herramientas (p. ej. desde conda-forge/bioconda) sin modificar el sistema base.
- **Lmod (Environment Modules)** como una capa de “menú” para seleccionar versiones con comandos simples (`module load`, `module swap`), sin que los usuarios tengan que aprender rutas internas ni activar entornos manualmente. En este servidor, Lmod se utiliza para exponer versiones que internamente pueden provenir de:
 - **Micromamba** (entornos versionados): ideal para herramientas de uso común y rápida instalación.
 - **Apptainer** (imágenes `.sif`): ideal para pipelines complejos reproducibles o dependencias difíciles.
 - (Opcional) **instalaciones nativas** en `/opt` para software compilado a mano cuando aplique.

La idea central es que **Lmod no instala software**: únicamente configura el ambiente (`PATH` y variables) para apuntar a una versión específica de manera controlada y consistente, mientras

que Micromamba y los contenedores proporcionan el software y sus dependencias. Este enfoque reduce conflictos entre proyectos, facilita el mantenimiento y fortalece la reproducibilidad científica.

5.2 Estado actual: Docker y Podman instalados.

- Docker se utilizará principalmente para **servicios administrados** (p. ej. herramientas internas, dashboards, servicios web).
- Podman se utilizará como alternativa **rootless** para ejecutar contenedores sin daemon, cuando se requiera un modelo más seguro o aislado.

5.2.1 Versiones y verificación

```
docker --version
podman --version
```

```
stoich@maginot:~$ docker --version
Docker version 29.1.3, build f52814d
stoich@maginot:~$ podman --version
podman version 5.6.0
```

5.2.2 Docker (prueba rápida, sin construir)

```
docker run --rm alpine:latest cat /etc/os-release
```

```
stoich@maginot:~$ docker run --rm alpine:latest cat /etc/os-release
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.23.2
PRETTY_NAME="Alpine Linux v3.23"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/aports/-/issues"
stoich@maginot:~$
```

5.2.3 Podman (prueba rápida, sin construir)

```
podman run --rm docker.io/library/alpine:latest cat /etc/os-release
```

```
stoich@maginot:~$ podman run --rm docker.io/library/alpine:latest cat /etc/os-release
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.23.2
PRETTY_NAME="Alpine Linux v3.23"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/aports/-/issues"
stoich@maginot:~$
```

5.3 Apptainer (+ fakeroot)

Instalación:

```
sudo dnf install -y apptainer fakeroot fakeroot-libs
```

5.3.1 Verifica que ya existe el comando

```
apptainer --version
which apptainer
rpm -q apptainer fakeroot
```

```
stoich@maginot:~$ apptainer --version
apptainer version 1.4.5-2.el10_1
stoich@maginot:~$ which apptainer
/usr/bin/apptainer
stoich@maginot:~$ rpm -q apptainer fakeroot
apptainer-1.4.5-2.el10_1.x86_64
fakeroot-1.37-1.el10_0.x86_64
stoich@maginot:~$
```

5.3.2 Prueba rápida (sin construir nada)

```
apptainer exec docker://alpine:latest cat /etc/os-release
```

```
stoich@maginot:~$ apptainer exec docker://alpine:latest cat /etc/os-release
INFO:   Converting OCI blobs to SIF format
INFO:   Starting build...
INFO:   Fetching OCI image...
3.7MiB / 3.7MiB [=====] 100 %0s
INFO:   Extracting OCI image...
INFO:   Inserting Apptainer configuration...
INFO:   Creating SIF file...
[=====] 100 % 0s
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.23.3
PRETTY_NAME="Alpine Linux v3.23"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/aports/-/issues"
stoich@maginot:~$
```

5.4 Micromamba

5.4.1 Descargar el binario e instalarlo en /usr/local/bin:

```
sudo mkdir -p /usr/local/lib/micromamba
cd /tmp

curl -Ls https://micro.mamba.pm/api/micromamba/linux-64/latest \
  | tar -xvj bin/micromamba

sudo mv bin/micromamba /usr/local/lib/micromamba/micromamba
sudo ln -sf /usr/local/lib/micromamba/micromamba /usr/local/bin/micromamba
sudo chmod 755 /usr/local/lib/micromamba/micromamba
```

5.4.2 Verificación

```
micromamba --version
which micromamba
```

5.4.3 Prefijo global (ambientes compartidos) y permisos “solo root escribe”

```
sudo mkdir -p /opt/micromamba/{envs,pkgs}
sudo chown -R root:root /opt/micromamba
sudo chmod -R 755 /opt/micromamba
```

Con esto:

- usuarios pueden **leer/usar** ambientes en /opt/micromamba
- usuarios **no pueden crear/modificar** ambientes ahí

5.4.4 Configuración global (canales y prioridades)

```
sudo mkdir -p /etc/micromamba

sudo tee /etc/micromamba/condarc >/dev/null <<'EOF'
channels:
  - conda-forge
  - bioconda
  - defaults
channel_priority: strict
auto_activate_base: false
EOF

sudo chmod 644 /etc/micromamba/condarc
```

5.4.5 Habilitar para todos los usuarios (activate/run) vía /etc/profile.d

```
sudo tee /etc/profile.d/micromamba.sh >/dev/null <<'EOF'
# Micromamba global (servidor)
export MAMBA_ROOT_PREFIX=/opt/micromamba
export CONDARC=/etc/micromamba/condarc
export MAMBA_EXE=/usr/local/bin/micromamba

# Habilita: micromamba activate / deactivate
eval "$("$MAMBA_EXE" shell hook -s bash)"
EOF

sudo chmod 644 /etc/profile.d/micromamba.sh
```

5.4.6 Aplicar en sesión actual (o cerrar sesión y re-entrar)

```
source /etc/profile.d/micromamba.sh
```

5.4.7 Crear ambientes e instalar paquetes (solo como root)

Ejemplo de creación:

```
micromamba create -n bioinfo -y \
  samtools=1.19.* mafft fastqc
```

Listar:

```
sudo micromamba env list
sudo micromamba list -n bioinfo
```

Instalar algo adicional:

```
micromamba activate bioinfo
sudo micromamba install -n bioinfo -y iqtree
```

5.4.8 Usar el ambiente desde un script

```
#!/usr/bin/env bash
set -euo pipefail

# Asegura variables globales aunque el script corra sin login shell
source /etc/profile.d/micromamba.sh

micromamba run -n bioinfo -- samtools --version
micromamba run -n bioinfo -- mafft --version
```

5.4.9 Reproducibilidad (lock del ambiente)

Export exacto (lista explícita de paquetes):

```
sudo micromamba env export -n bioinfo --explicit > bioinfo-linux-64.lock
```

Recrear idéntico desde lock:

```
sudo micromamba create -n bioinfo_v2 --file bioinfo-linux-64.lock -y
```

5.5 Multiversión de software (Lmod / Environment Modules)

5.5.1 Instalar Lmod en Rocky (EPEL + CRB)

```
sudo dnf install -y Lmod
```

Verificación:

```
rpm -q Lmod  
bash -lc 'module --version || ml --version'
```

```
stoich@maginot:~$ rpm -q Lmod  
Lmod-8.7.65-2.el10_1.x86_64  
stoich@maginot:~$ bash -lc 'module --version || ml --version'  
  
Modules based on Lua: Version 8.7.65 2025-08-05 10:24 -06:00  
by Robert McLay mclay@tacc.utexas.edu  
  
stoich@maginot:~$
```

5.5.2 Asegurar que module funcione para *todos* (login y no-login)

El paquete de Lmod instala scripts de inicialización en /etc/profile.d/ (p. ej. z00_lmod.sh) que cargan el init de Lmod.

Prueba en una sesión nueva (login shell):

```
module avail | head
```

```
stoich@maginot:~$ module avail | head  
  
----- /usr/share/lmod/lmod/modulefiles/Core -----  
lmod      settarg  
  
If the avail list is too long consider trying:  
  
"module --default avail" or "ml -d av" to just list the default modules.  
"module overview" or "ml ov" to display the number of modules for each name.  
  
Use "module spider" to find all possible modules and extensions.  
Use "module keyword key1 key2 ..." to search for all possible modules matching any of the "keys".  
  
stoich@maginot:~$
```

5.5.3 Árbol local de modulefiles y permisos

Se creó un árbol de modulefiles local en /opt/modulefiles:

```
sudo mkdir -p /opt/modulefiles/{bio,pipelines,core}
sudo chown -R root:root /opt/modulefiles
sudo chmod -R 755 /opt/modulefiles
```

Permisos recomendados:

- directorios: 755 (todos pueden listar/entrar)
- archivos .lua: 644 (todos pueden leer; solo root escribe)

5.5.4 Hacer que Lmod *siempre* vea /opt/modulefiles

Crear un script que corra al inicio de sesión y agregue ese path al MODULEPATH:

```
sudo tee /etc/profile.d/z10_modulepath.sh >/dev/null <<'EOF'
# Site modulefiles (Lmod)
if type module >/dev/null 2>&1; then
    module use --append /opt/modulefiles/bio
    module use --append /opt/modulefiles/pipelines
    module use --append /opt/modulefiles/core
fi
EOF
sudo chmod 644 /etc/profile.d/z10_modulepath.sh
```

Nota: abre una nueva sesión SSH (o recarga con `source /etc/profile.d/z10_modulepath.sh`) para que aplique.

5.5.5 Publicar “programas” a usuarios mediante módulos

Micromamba (envs versionados) + Lmod

Ejemplo: si existe un env en:

- /opt/micromamba/envs/samtools-1.19

Crear el modulefile:


```

sudo mkdir -p /opt/modulefiles/bio/samtools
sudo tee /opt/modulefiles/bio/samtools/1.19.lua >/dev/null <<'EOF'
help([[
Samtools 1.19 (Micromamba env)
]])
whatis("Samtools 1.19 via Micromamba")
conflict("samtools")

local envdir = "/opt/micromamba/envs/samtools-1.19"
prepend_path("PATH", pathJoin(envdir, "bin"))
prepend_path("LD_LIBRARY_PATH", pathJoin(envdir, "lib"))
setenv("SAMTOOLS_ROOT", envdir)
EOF
sudo chmod 644 /opt/modulefiles/bio/samtools/1.19.lua

```

Uso por parte del usuario:

```

module load samtools/1.19
samtools --version

```

5.5.6 Apptainer (.sif) + Lmod (envolver comando)

Directorio de contenedores:

```

sudo mkdir -p /opt/containers
sudo chown -R root:root /opt/containers
sudo find /opt/containers -type d -exec chmod 755 {} \;
sudo find /opt/containers -type f -name "*.sif" -exec chmod 644 {} \;

```

Ejemplo: imagen en:

- /opt/containers/revbayes/rb_v125_mpi/revbayes_v125_mpi.sif

Crear modulefile:

```

sudo mkdir -p /opt/modulefiles/pipelines/revbayes

sudo tee /opt/modulefiles/pipelines/revbayes/1.2.5-mpi.lua >/dev/null <<'EOF'
help([[
RevBayes 1.2.5 (MPI) via Apptainer.

Este módulo expone:
- rb-mpi      : ejecuta rb-mpi (1 proceso) en el directorio actual.
- rb-mpi-np N : ejecuta rb-mpi con mpirun -np N (multinúcleo) en el directorio
                actual.

```

```

Ejemplo típico:
  cd /ruta/al/proyecto/Clade_001/input
  module load revbayes/1.2.5-mpi
  rb-mpi-np 8 Clade_001.rev

Binds extra (por ejemplo /srv):
  export APPTAINER_BINDPATH="/srv:/srv"
]])
whatis("RevBayes 1.2.5 MPI (Apptainer)")
conflict("revbayes")

local img = "/opt/containers/revbayes/rb_v125_mpi/revbayes_v125_mpi.sif"
setenv("REVBAYES_SIF", img)

-- 1) Wrapper mínimo (1 proceso), ejecuta en /work (= $PWD del host)
set_shell_function("rb-mpi",
  "apptainer exec --cleanenv --bind \"\$PWD\":/work --pwd /work " .. img .. " rb-mpi
  \"\$@\"",
  "true")

-- 2) Wrapper multinúcleo: primer argumento = N procesos
set_shell_function("rb-mpi-np",
  "N=${1:-1}; shift; apptainer exec --cleanenv --bind \"\$PWD\":/work --pwd /work "
  .. img .. " mpirun -np $N rb-mpi \"\$@\"",
  "true")
EOF

sudo chmod 644 /opt/modulefiles/pipelines/revbayes/1.2.5-mpi.lua

```

Uso por parte del usuario:

```

module load revbayes/1.2.5-mpi
rb-mpi --help | head

```

5.5.7 Nota importante: evita conflictos con environment-modules

En Rocky puede coexistir `environment-modules` y `Lmod`; sin embargo, si ambos están instalados, el comando `module` puede apuntar al “equivocado” (`alternatives`). Para evitar confusiones, se recomienda **estandarizar** el uso de `Lmod` (y evitar instalar `environment-modules` si no es necesario).

5.6 Buenas prácticas de reproducibilidad

- Registrar el estado del ambiente en salidas y bitácoras (`module list`, `micromamba env list`).
- Para entornos con Micromamba, conservar archivos `--explicit (*.lock)` para recreación idéntica.
- Para contenedores, registrar **ruta exacta**, versión y/o hash de la imagen `.sif`.
- Evitar instalaciones globales “ad-hoc” sin documentación (preferir módulos, envs o contenedores versionados).

5.6.1 Checklist final (uso por usuarios)

Como administrador (verificación básica):

```
sudo -i  
bash -lc 'module avail | head'
```

```

root@maginot:~# bash -lc 'module avail | head'

----- /usr/share/lmod/lmod/modulefiles/Core -----
lmod      settarg

----- /opt/modulefiles/bio -----

samtools/1.19

----- /opt/modulefiles/pipelines -----

amas/1.0          biopython/1.86      hammer/3.4          repeatmodeler/2.0.7
astral/5.7.8      blast/2.17.0        orthofinder/3.1.2   revbayes/1.2.5-mpi
augustus/3.3.3    busco/6.0.0         phyloCFs/1.0        salmon/1.10.3
augustus/3.5.0 (D) deepte/1.0          phylonetworks/1.0   seqtools/1.40.0
beast/2.6.2       edta/2.0.1          raxml/8.2.12        transdecoder/5.7.1
beast/2.7.7 (D)   edta/2.2.2 (D)     repeatmasker/4.2.2   trinity/2.15.2

Where:
D: Default Module

If the avail list is too long consider trying:

"module --default avail" or "ml -d av" to just list the default modules.
"module overview" or "ml ov" to display the number of modules for each name.

Use "module spider" to find all possible modules and extensions.
Use "module keyword key1 key2 ..." to search for all possible modules matching any of the
"keys".

```

Como usuario (sesión nueva):

module spider samtools

```

module avail | head
module spider samtools

```

```
stoich@maginot:~$ module avail | head

----- /usr/share/lmod/lmod/modulefiles/Core -----
lmod      settarg

----- /opt/modulefiles/bio -----
samtools/1.19

----- /opt/modulefiles/pipelines -----
amas/1.8      augustus/3.5.0 (D)  biopython/1.86  deept/1.8      hammer/3.4      phylone/works/1.0  repeatmodeler/2.0.7  segtools/1.40.0
astral/5.7.8  beast/2.6.2        blast/2.17.0    edta/2.0.1     orthofinder/3.1.2  raxml/8.2.12      revbayes/1.2.5-mpi  transdecoder/5.7.1
augustus/3.3.3  beast/2.7.7 (D)  busco/6.0.0     edta/2.2.2 (D)  phyloCFS/1.0      repeatmasker/4.2.2  salmon/1.10.3        trinity/2.15.2

Where:
D: Default Module

If the avail list is too long consider trying:

"module --default avail" or "ml -d av" to just list the default modules.
"module overview" or "ml ov" to display the number of modules for each name.

Use "module spider" to find all possible modules and extensions.
Use "module keyword key1 key2 ..." to search for all possible modules matching any of the "keys".

stoich@maginot:~$ module spider samtools

-----
samtools: samtools/1.19

-----

This module can be loaded directly: module load samtools/1.19

Help:
  Samtools 1.19 (Micromamba env)
```

module spider es útil para localizar módulos cuando el árbol crece en número de versiones.

6 Servidor de archivos con Samba

6.1 Contexto y objetivo

Con el fin de contar con un **servidor de archivos compatible con macOS**, se configuró en MAGINOT un recurso compartido mediante **Samba (SMB)** sobre **Rocky Linux 10.1**.

La elección de Samba se debe a que **macOS se conecta de forma nativa mediante smb://**, lo que permite acceder fácilmente a carpetas compartidas desde Finder.

En este caso, se configuró la carpeta:

```
/srv/bishop
```

como un recurso privado para el usuario **stoich**.

6.2 Alcance de la configuración

- Sistema operativo del servidor: **Rocky Linux 10.1**
- Protocolo de compartición: **SMB/Samba**
- Carpeta compartida: **/srv/bishop**
- Usuario autorizado: **stoich**
- Cliente principal: **macOS**
- Acceso previsto: **red local** o mediante mecanismos internos de red

6.3 Preparación de la carpeta compartida

Primero se verificó que la carpeta existiera dentro de **/srv/** y se definió como propiedad exclusiva del usuario **stoich**.

6.3.1 Establecer propietario

```
sudo chown -R stoich:stoich /srv/bishop
```

6.3.2 Restringir permisos

Para evitar acceso de otros usuarios del sistema, se dejaron permisos privados:

```
sudo chmod 700 /srv/bishop sudo find /srv/bishop -type d -exec chmod 700 {} \; sudo  
find /srv/bishop -type f -exec chmod 600 {} \;
```

Con ello:

- los directorios quedan con permisos 700
- los archivos quedan con permisos 600

6.3.3 Instalación de Samba

Se instaló Samba y las utilerías de cliente:

```
sudo dnf install -y samba samba-client
```

6.4 Configuración de SELinux

Dado que Rocky Linux utiliza **SELinux**, fue necesario etiquetar la carpeta compartida con el contexto apropiado para Samba.

6.4.1 Instalar semanage si es necesario

```
sudo dnf install -y polycoreutils-python-utils
```

6.4.2 Asignar contexto SELinux

```
sudo semanage fcontext -a -t samba_share_t "/srv/bishop(/.*)" sudo restorecon -Rv  
/srv/bishop
```

Esto permite que Samba pueda acceder a la carpeta sin desactivar SELinux.

6.5 Configuración del usuario Samba

Aunque el usuario `stoich` ya existía en Linux, Samba requiere registrar su propia contraseña.

```
sudo smbpasswd -a stoich sudo smbpasswd -e stoich
```

6.6 Configuración de `/etc/samba/smb.conf`

Se utilizó el archivo de configuración por defecto de Rocky Linux y se añadieron únicamente los ajustes necesarios para compatibilidad con macOS y para exponer el recurso `bishop`.

6.6.1 Archivo de configuración

```
[global]
    workgroup = SAMBA
    security = user
    vfs objects = fruit streams_xattr

    passdb backend = tdbsam

    printing = cups
    printcap name = cups
    load printers = yes
    cups options = raw

    # Install samba-usershares package for support
    include = /etc/samba/usershares.conf

[homes]
    comment = Home Directories
    valid users = %S, %D%w%S
    browseable = No
    read only = No
    inherit acls = Yes

[printers]
    comment = All Printers
    path = /var/tmp
    printable = Yes
    create mask = 0600
    browseable = No

[print$]
```



```

comment = Printer Drivers
path = /var/lib/samba/drivers
write list = printadmin root
force group = printadmin
create mask = 0664
directory mask = 0775

[bishop]
path = /srv/bishop
browseable = yes
read only = no
guest ok = no
valid users = stoich
force user = stoich
create mask = 0600
directory mask = 0700

```

6.6.2 Notas sobre esta configuración

- `vfs objects = fruit streams_xattr` mejora la compatibilidad con clientes **macOS**
- `guest ok = no` evita accesos anónimos
- `valid users = stoich` restringe el acceso solo al usuario autorizado
- `force user = stoich` asegura que las operaciones dentro del recurso se realicen como ese usuario

6.7 Validación de la configuración

Antes de iniciar el servicio, se verificó que el archivo no tuviera errores de sintaxis:

```
sudo testparm
```

Una salida exitosa indica que la configuración es válida.

6.8 Apertura del servicio en el firewall

Para permitir conexiones SMB dentro de la red, se habilitó el servicio Samba en `firewalld`:

```
sudo firewall-cmd --permanent --add-service=samba sudo firewall-cmd --reload
```

6.9 Activación de los servicios

Se habilitaron e iniciaron los demonios de Samba:

```
sudo systemctl enable --now smb nmb
```

6.9.1 Verificación de estado

```
systemctl status smb nmb
```

La salida esperada debe mostrar ambos servicios como:

```
active (running)
```

6.10 Prueba local en el servidor

Para verificar que el recurso estuviera disponible correctamente, se listaron los recursos Samba desde el propio servidor:

```
smbclient -L localhost -U stoich
```

Con ello debe aparecer el recurso compartido `bishop`.

6.11 Conexión desde macOS

Desde Finder, el recurso se puede montar mediante:

Ir → Conectarse al servidor

y utilizar una de las siguientes rutas:

```
smb://IP_DE_MAGINOT/bishop
```

o bien:

```
smb://maginot/bishop
```

Después se autentica con:

- **usuario:** stoich
- **contraseña:** la definida con `smbpasswd`

7 Monitoreo y Observabilidad

7.1 Contexto y objetivo

Este servidor Rocky Linux incluye **Cockpit** (Web Console) como interfaz web de administración. Además, se desea montar un stack de monitoreo clásico basado en:

- **Node Exporter**: expone métricas del sistema (CPU, RAM, disco, red, etc.) vía HTTP.
- **Prometheus**: recolecta métricas periódicamente, las almacena en una base de datos de series de tiempo (TSDB) y permite consultas con PromQL.
- **Grafana**: consume Prometheus como fuente de datos y permite crear tableros (dashboards) y alertas.

Objetivo: habilitar un sistema de monitoreo y observabilidad local (en el mismo servidor), manteniendo **Cockpit** como herramienta adicional de administración/diagnóstico, y configurando los puertos sin colisiones, con buenas prácticas de seguridad.

7.2 Node Exporter (métricas del host) en :9100

7.2.1 Crear usuario y directorios

```
sudo useradd --no-create-home --shell /sbin/nologin node_exporter 2>/dev/null || true
sudo mkdir -p /var/lib/node_exporter/textfile_collector
sudo chown -R node_exporter:node_exporter /var/lib/node_exporter
```

7.2.2 Descargar e instalar el binario

```
cd /tmp
NE_VER="1.10.2"
NE_FILE="node_exporter-${NE_VER}.linux-amd64.tar.gz"

wget -O "${NE_FILE}"
    "https://github.com/prometheus/node_exporter/releases/download/v${NE_VER}/${NE_FILE}"

tar -xzf "${NE_FILE}"
```

```
sudo install -m 0755 "node_exporter-${NE_VER}.linux-amd64/node_exporter"  
/usr/local/bin/node_exporter
```

7.2.3 Crear servicio systemd

```
sudo tee /etc/systemd/system/node_exporter.service >/dev/null <<'EOF'  
[Unit]  
Description=Prometheus Node Exporter  
Wants=network-online.target  
After=network-online.target  
  
[Service]  
User=node_exporter  
Group=node_exporter  
Type=simple  
ExecStart=/usr/local/bin/node_exporter \  
    --web.listen-address=:9100 \  
    --collector.textfile.directory=/var/lib/node_exporter/textfile_collector  
Restart=on-failure  
  
[Install]  
WantedBy=multi-user.target  
EOF  
  
sudo systemctl daemon-reload  
sudo systemctl enable --now node_exporter
```

7.2.4 Prueba local (métricas):

```
sudo systemctl status node_exporter --no-pager -l  
curl -s http://127.0.0.1:9100/metrics | head
```

```

stoich@maginot:~$ sudo systemctl status node_exporter --no-pager -l
● node_exporter.service - Prometheus Node Exporter
   Loaded: loaded (/etc/systemd/system/node_exporter.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-02-01 11:24:13 CST; 2 weeks 1 day ago
     Invocation: 31d6e22e699c42c5bd74d4753c975d50
   Main PID: 3280 (node_exporter)
      Tasks: 5 (limit: 1645517)
     Memory: 34.8M (peak: 40.9M)
        CPU: 2h 46min 45.777s
    CGroup: /system.slice/node_exporter.service
            └─3280 /usr/local/bin/node_exporter --web.listen-address=:9100 --collector.textfile.directory=/var/lib/node_exporter/textfile_collec
tor

Feb 01 11:24:14 maginot node_exporter[3280]: time=2026-02-01T11:24:14.030-06:00 level=INFO source=node_exporter.go:141 msg=time
Feb 01 11:24:14 maginot node_exporter[3280]: time=2026-02-01T11:24:14.030-06:00 level=INFO source=node_exporter.go:141 msg=timex
Feb 01 11:24:14 maginot node_exporter[3280]: time=2026-02-01T11:24:14.030-06:00 level=INFO source=node_exporter.go:141 msg=udp_queues
Feb 01 11:24:14 maginot node_exporter[3280]: time=2026-02-01T11:24:14.030-06:00 level=INFO source=node_exporter.go:141 msg=uname
Feb 01 11:24:14 maginot node_exporter[3280]: time=2026-02-01T11:24:14.030-06:00 level=INFO source=node_exporter.go:141 msg=vmstat
Feb 01 11:24:14 maginot node_exporter[3280]: time=2026-02-01T11:24:14.030-06:00 level=INFO source=node_exporter.go:141 msg=watchdog
Feb 01 11:24:14 maginot node_exporter[3280]: time=2026-02-01T11:24:14.030-06:00 level=INFO source=node_exporter.go:141 msg=xfs
Feb 01 11:24:14 maginot node_exporter[3280]: time=2026-02-01T11:24:14.030-06:00 level=INFO source=node_exporter.go:141 msg=zfs
Feb 01 11:24:14 maginot node_exporter[3280]: time=2026-02-01T11:24:14.031-06:00 level=INFO source=tls_config.go:346 msg="Listening on" address=[
:]:9100
Feb 01 11:24:14 maginot node_exporter[3280]: time=2026-02-01T11:24:14.032-06:00 level=INFO source=tls_config.go:349 msg="TLS is disabled." http2=
false address=[:]:9100
stoich@maginot:~$ curl -s http://127.0.0.1:9100/metrics | head
# HELP go_gc_duration_seconds A summary of the wall-time pause (stop-the-world) duration in garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 1.5398e-05
go_gc_duration_seconds{quantile="0.25"} 2.5668e-05
go_gc_duration_seconds{quantile="0.5"} 3.1979e-05
go_gc_duration_seconds{quantile="0.75"} 4.668e-05
go_gc_duration_seconds{quantile="1"} 8.9882e-05
go_gc_duration_seconds_sum 14.376130381
go_gc_duration_seconds_count 351750
# HELP go_gc_gogc_percent Heap size target percentage configured by the user, otherwise 100. This value is set by the GOGC environment variable,
and the runtime/debug.SetGCPercent function. Sourced from /gc/gogc:percent.
stoich@maginot:~$

```

7.3 Prometheus (TSDB + UI) en :9095

7.3.1 Crear usuario y directorios

```

sudo useradd --no-create-home --shell /sbin/nologin prometheus 2>/dev/null || true
sudo mkdir -p /etc/prometheus /var/lib/prometheus
sudo chown -R prometheus:prometheus /etc/prometheus /var/lib/prometheus

```

7.3.2 Descargar e instalar Prometheus

```

cd /tmp
P_VER="3.9.1"
P_FILE="prometheus-${P_VER}.linux-amd64.tar.gz"

wget -O "${P_FILE}"
"https://github.com/prometheus/prometheus/releases/download/v${P_VER}/${P_FILE}"
tar -xzf "${P_FILE}"

```

```
sudo install -m 0755 "prometheus-${P_VER}.linux-amd64/prometheus"  
    /usr/local/bin/prometheus  
sudo install -m 0755 "prometheus-${P_VER}.linux-amd64/promtool"  
    /usr/local/bin/promtool
```

7.3.3 Configuración prometheus.yml

```
sudo tee /etc/prometheus/prometheus.yml >/dev/null <<'EOF'  
global:  
  scrape_interval: 15s  
  evaluation_interval: 15s  
  
scrape_configs:  
  - job_name: "prometheus"  
    static_configs:  
      - targets: ["127.0.0.1:9095"]  
  
  - job_name: "node"  
    static_configs:  
      - targets: ["127.0.0.1:9100"]  
EOF  
  
sudo chown prometheus:prometheus /etc/prometheus/prometheus.yml  
sudo -u prometheus /usr/local/bin/promtool check config  
    /etc/prometheus/prometheus.yml
```

7.3.4 Servicio systemd (Prometheus en 9095)

Se omiten `--web.console.templates` y `--web.console.libraries` para evitar referencias a rutas no utilizadas.

```
sudo tee /etc/systemd/system/prometheus.service >/dev/null <<'EOF'  
[Unit]  
Description=Prometheus  
Wants=network-online.target  
After=network-online.target  
  
[Service]  
User=prometheus  
Group=prometheus  
Type=simple  
ExecStart=/usr/local/bin/prometheus \  
    --config.file=/etc/prometheus/prometheus.yml \  
    --web.console.templates=/etc/prometheus/console.templates \  
    --web.console.libraries=/etc/prometheus/console.libraries
```

```

--storage.tsdb.path=/var/lib/prometheus \
--web.listen-address=:9095 \
--web.console.templates=/etc/prometheus/consoles \
--web.console.libraries=/etc/prometheus/console_libraries
Restart=on-failure

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now prometheus

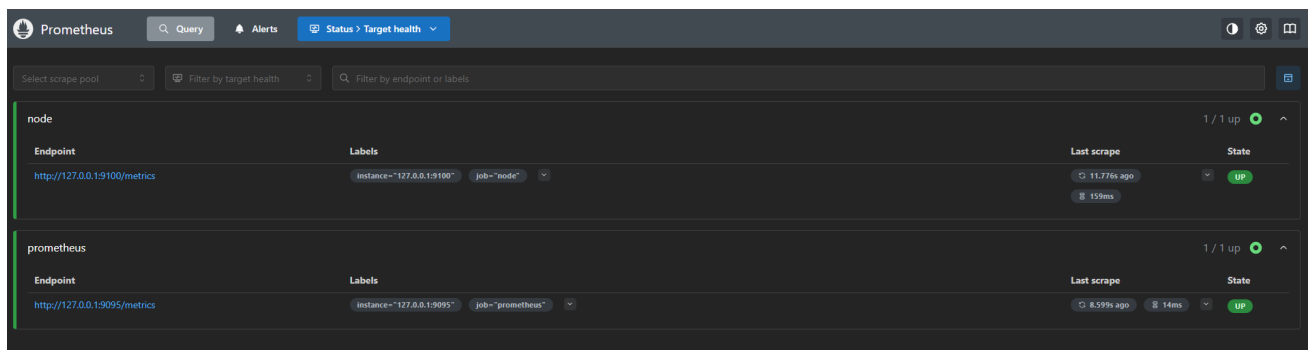
```

7.3.5 Prueba local

```
curl -s http://127.0.0.1:9095/-/ready && echo
```

Abrir en navegador:

- <http://127.0.0.1:9095> → Prometheus
- <http://127.0.0.1:9095/targets> (o “Status → Targets”) debe mostrar prometheus y node en **UP**.



7.4 Grafana (dashboards)

7.4.1 Agregar repositorio e instalar (repo oficial RPM)

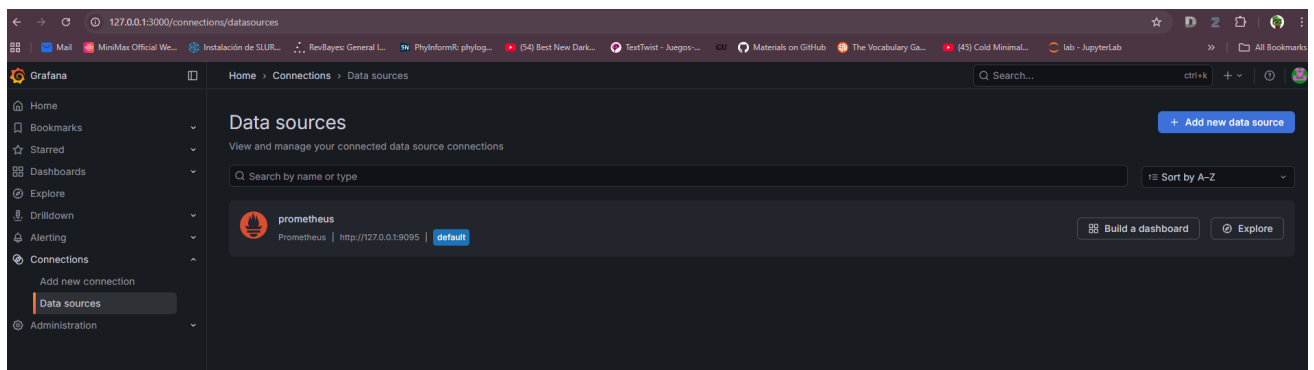
```
wget -q -O gpg.key https://rpm.grafana.com/gpg.key
sudo rpm --import gpg.key

sudo tee /etc/yum.repos.d/grafana.repo >/dev/null <<'EOF'
[grafana]
name=grafana
baseurl=https://rpm.grafana.com
repo_gpgcheck=1
enabled=1
gpgcheck=1
gpgkey=https://rpm.grafana.com/gpg.key
sslverify=1
sslcacert=/etc/pki/tls/certs/ca-bundle.crt
EOF

sudo dnf -y install grafana
sudo systemctl enable --now grafana-server
```

7.4.2 Conectar Grafana a Prometheus

- Entrar a `http://127.0.0.1:3000`
- Data Source: Prometheus
 - URL: `http://127.0.0.1:9095`
 - Save & test



7.4.3 Dashboards y paneles implementados

En Grafana se importó un tablero base tipo “**Node Exporter Full**” como referencia y se añadieron paneles específicos para MAGINOT (por recursos y puntos de montaje relevantes).

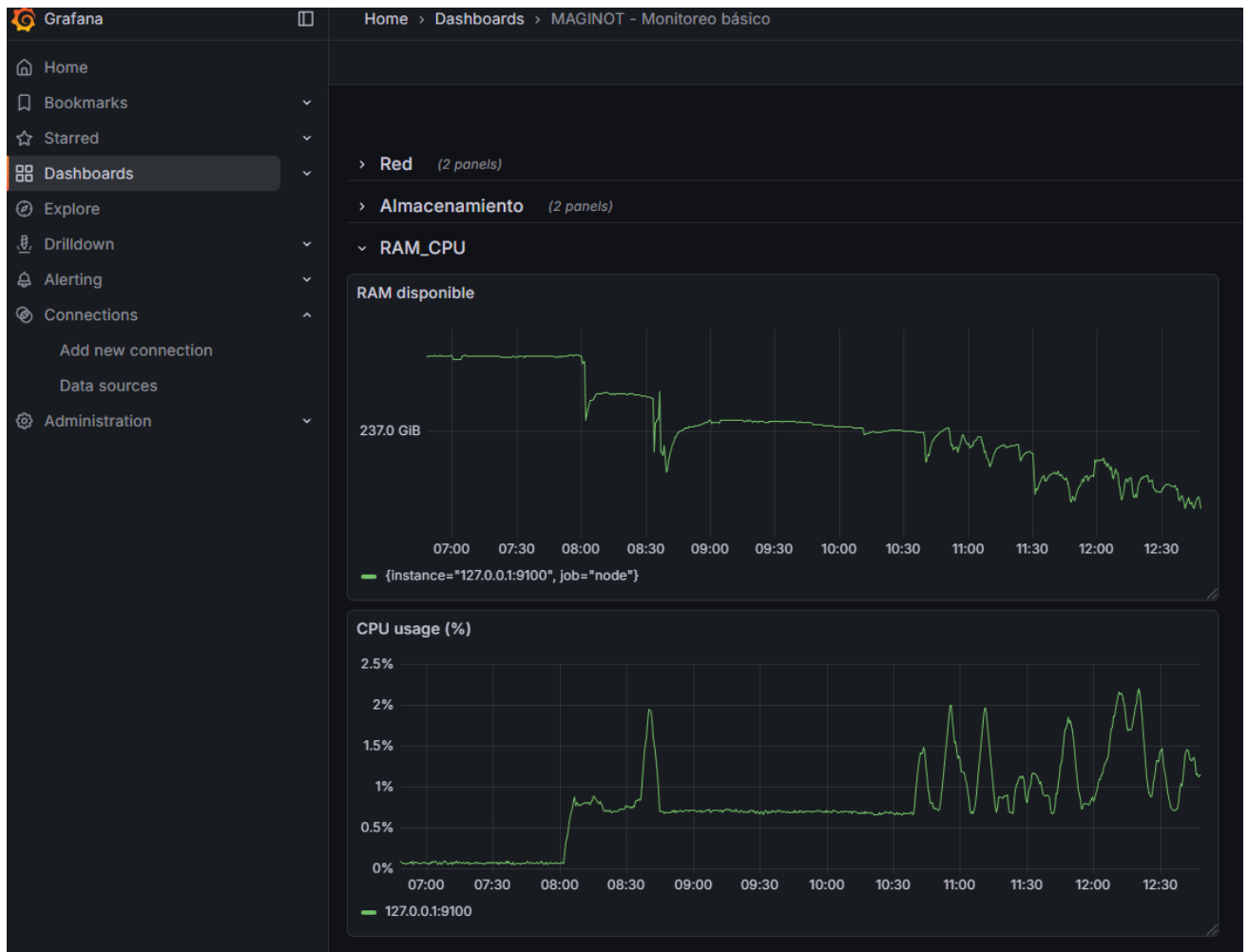
7.4.3.1 Row 1: RAM / CPU

RAM disponible (GiB):

```
node_memory_MemAvailable_bytes / (1024*1024*1024)
```

CPU en uso (%):

```
100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m]))) * 100)
```



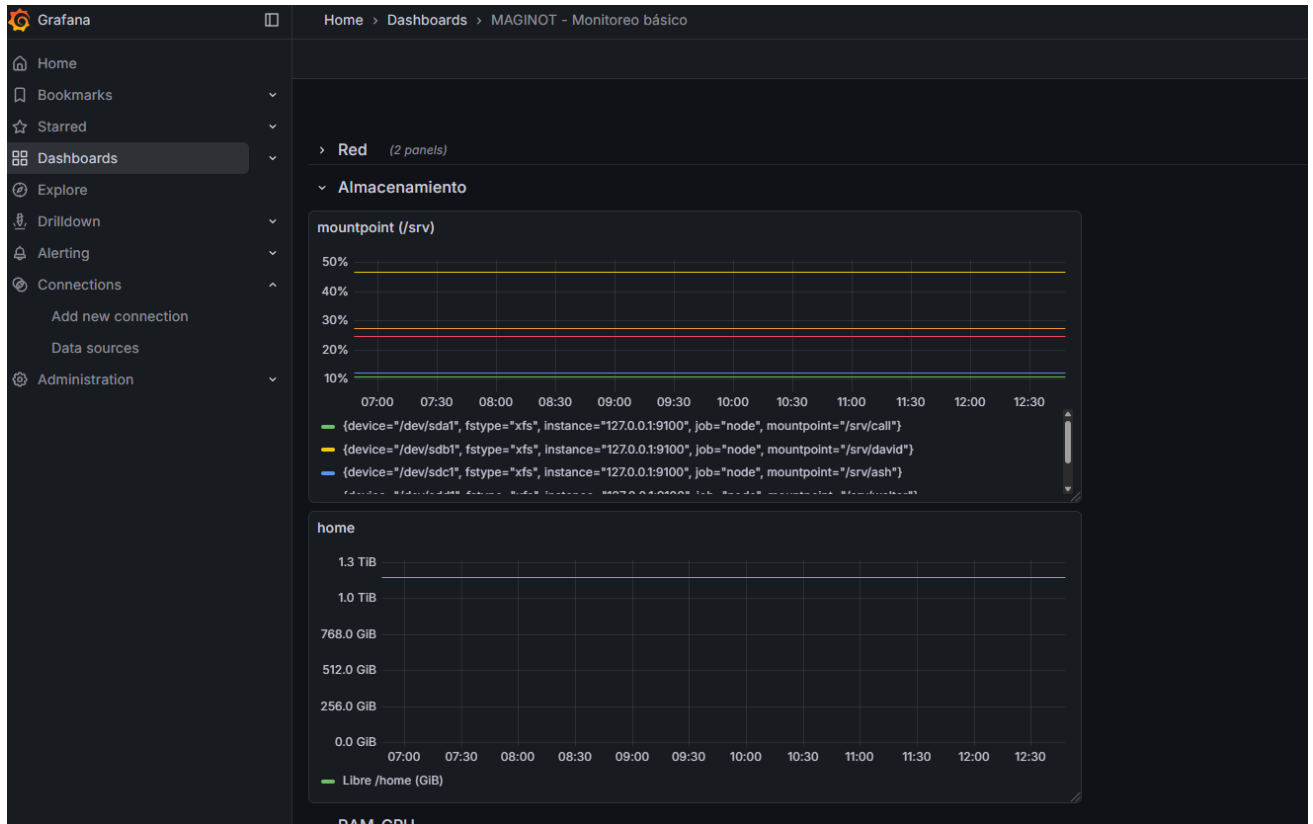
7.4.3.2 Row 2: Almacenamiento

Uso (%) por volúmenes en /srv/<volumen>:

```
100 * (1 - (
  node_filesystem_avail_bytes{mountpoint=~"/srv/[^/]+$",
    fstype!~"tmpfs|overlay|squashfs|devtmpfs"}
/
  node_filesystem_size_bytes{mountpoint=~"/srv/[^/]+$",
    fstype!~"tmpfs|overlay|squashfs|devtmpfs"}
))
```

Disponible (GiB) en /home:

```
node_filesystem_avail_bytes{mountpoint="/home",
    fstype!~"tmpfs|overlay|squashfs|devtmpfs"} / (1024*1024*1024)
```



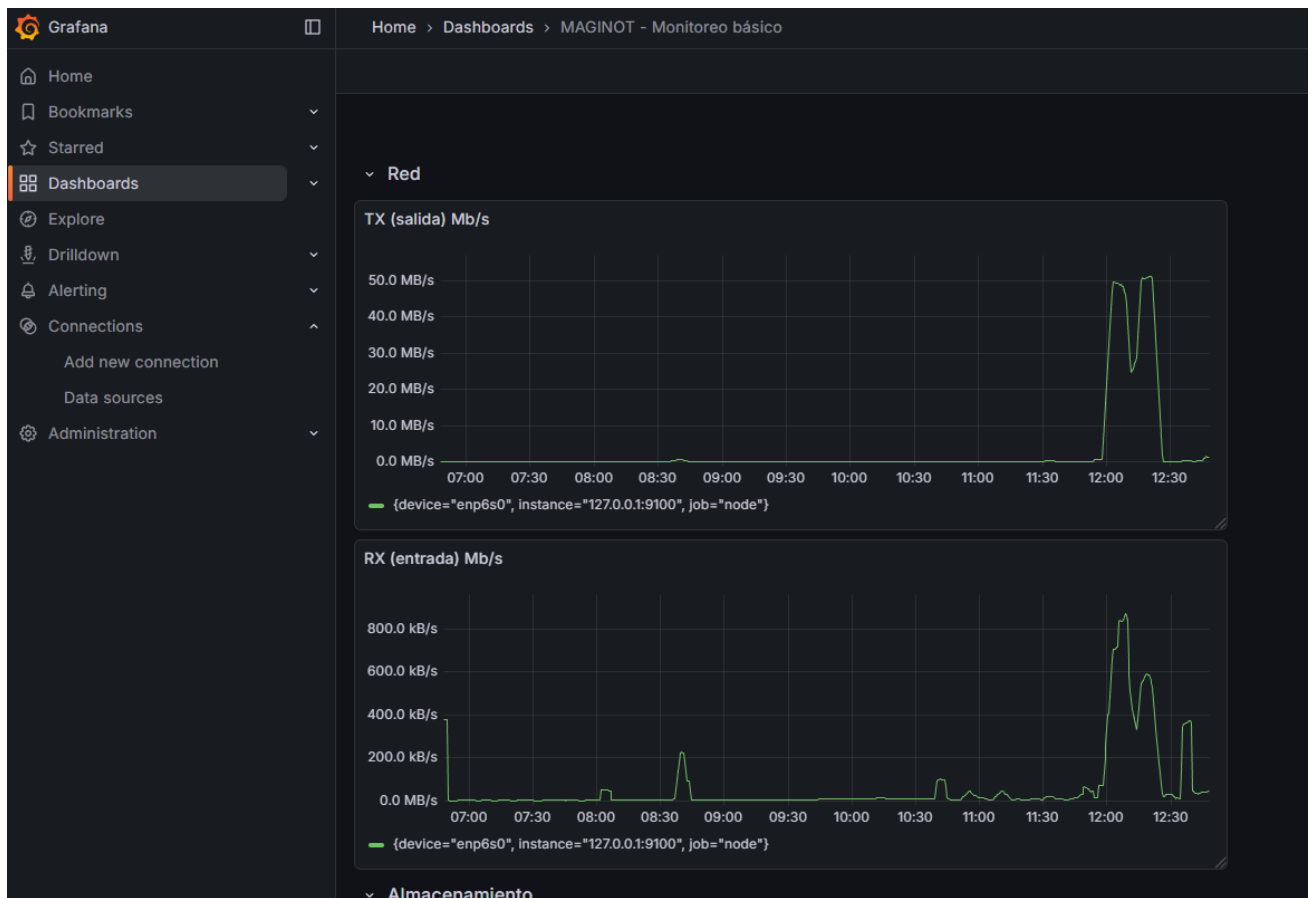
7.4.3.3 Row 3: Red

Salida (Tx, Mbps):

```
8 * rate(node_network_transmit_bytes_total{device="enp6s0"}[5m]) / 1e6
```

Entrada (Rx, Mbps):

```
8 * rate(node_network_receive_bytes_total{device="enp6s0"}[5m]) / 1e6
```



Nota: Los paneles se configuraron con unidades (GiB, %, Mbps) y umbrales para visualizar estados de alerta (p. ej. uso de disco > 80%).

7.4.4 Firewall (abre sólo lo necesario)

```
sudo firewall-cmd --permanent --add-port=3000/tcp
sudo firewall-cmd --permanent --add-port=9095/tcp
sudo firewall-cmd --permanent --add-port=9100/tcp
sudo firewall-cmd --reload
```

8 Instalación de Slurm para una sola máquina

8.1 Configuración de un nodo local con partición riple, MUNGE y registros en /srv/bishop

Slurm, acrónimo de *Simple Linux Utility for Resource Management*, es un sistema de administración de trabajos utilizado en servidores de cómputo y clústeres de alto rendimiento.

Su función principal es organizar el uso de los recursos computacionales, como CPUs, memoria y nodos de cómputo. En lugar de que los usuarios ejecuten procesos directamente en el servidor, envían trabajos a una cola. Slurm decide cuándo ejecutarlos, cuántos recursos asignarles y registra información básica sobre su ejecución.

En un clúster grande, Slurm puede administrar muchos nodos. Sin embargo, también puede utilizarse en una sola máquina. Esto es útil cuando se desea controlar el uso de recursos, evitar que un usuario sature el servidor y enseñar buenas prácticas de cómputo reproducible.

8.2 Objetivo

El objetivo de este tutorial es instalar y configurar Slurm en un servidor con Rocky Linux 10.1 usando una sola máquina como controlador y nodo de cómputo.

La partición de Slurm se llamará:

```
ripley
```

Además, para evitar llenar la partición raíz /, los archivos persistentes de Slurm se almacenarán en:

```
/srv/bishop/slurm
```

8.3 Esquema general

La instalación tendrá los siguientes componentes:

- `munge`: servicio de autenticación usado por Slurm.
- `slurmctld`: controlador principal de Slurm.
- `slurmd`: daemon que ejecuta trabajos en el nodo de cómputo.
- `slurm.conf`: archivo principal de configuración.
- `cgroup.conf`: configuración para controlar recursos con cgroups v2.
- `jobcomp/filetxt`: registro básico de trabajos terminados.

8.4 Información del servidor

Antes de instalar Slurm, es importante conocer las características del servidor.

```
hostname
hostname -f
cat /etc/os-release
nproc
free -h
df -h
lsblk
getent ahostsv4 maginot
```

En este caso, el servidor reportó:

```
Hostname: maginot
Sistema operativo: Rocky Linux 10.1
CPUs: 72
RAM: 251 GiB
```

También se decidió usar `/srv/bishop` para almacenar los archivos persistentes de Slurm, ya que la raíz `/` tiene poco espacio disponible.

8.5 Preparar repositorios y dependencias

En Rocky Linux 10.1, Slurm no se encuentra directamente disponible en los repositorios base como paquete listo para instalar. Por ello, se construyeron paquetes RPM a partir del código fuente de Slurm.

Primero se instalan las dependencias necesarias:

```
sudo dnf update -y
sudo dnf install -y epel-release dnf-plugins-core
sudo dnf config-manager --set-enabled crb
```

Después:

```
sudo dnf install -y \
    autoconf automake dbus-devel libbpf-devel make mariadb-devel \
    munge munge-libs munge-devel \
    pam-devel perl-devel perl-ExtUtils-MakeMaker \
    python3 readline-devel rpm-build rpmdevtools wget tar gcc gcc-c++
```

8.6 Construcción de los RPMs de Slurm

Crear el árbol de construcción RPM:

```
rpmdev-setuptree
```

Descargar Slurm:

```
cd ~/rpmbuild/SOURCES
wget https://download.schedmd.com/slurm/slurm-25.05.5.tar.bz2
```

Construir los paquetes:

```
rpmbuild -ta ~/rpmbuild/SOURCES/slurm-25.05.5.tar.bz2
```

Al finalizar, los RPMs quedan en:

```
~/rpmbuild/RPMS/x86_64/
```

Los paquetes mínimos necesarios para una instalación de una sola máquina son:

```
slurm-25.05.5-1.el10.x86_64.rpm
slurm-slurmctld-25.05.5-1.el10.x86_64.rpm
slurm-slurmd-25.05.5-1.el10.x86_64.rpm
```

8.7 Instalación de Slurm

Instalar los RPMs generados:

```
sudo dnf install -y \  
  ~/rpmbuild/RPMS/x86_64/slurm-25.05.5-1.el10.x86_64.rpm \  
  ~/rpmbuild/RPMS/x86_64/slurm-slurmctld-25.05.5-1.el10.x86_64.rpm \  
  ~/rpmbuild/RPMS/x86_64/slurm-slurmd-25.05.5-1.el10.x86_64.rpm
```

8.8 Crear usuario y grupo de Slurm

Slurm debe ejecutarse con un usuario de sistema dedicado.

```
sudo groupadd -g 2002 slurm 2>/dev/null || true  
  
sudo useradd -u 2002 -g slurm -c "SLURM Workload Manager" \  
  -d /var/lib/slurm -s /sbin/nologin -m slurm 2>/dev/null || true
```

Verificar:

```
id slurm
```

8.9 Configurar MUNGE

MUNGE permite autenticar los componentes de Slurm.

Crear la llave:

```
sudo dd if=/dev/urandom bs=1 count=1024 of=/etc/munge/munge.key  
sudo chown munge:munge /etc/munge/munge.key  
sudo chmod 0400 /etc/munge/munge.key
```

Iniciar el servicio:

```
sudo systemctl enable --now munge  
systemctl status munge --no-pager -l
```

Probar MUNGE:

```
munge -n | unmunge | head
```


Si el resultado muestra:

```
STATUS: Success (0)
```

MUNGE está funcionando correctamente.

8.10 Crear directorios de Slurm en /srv/bishop

Para evitar llenar la partición raíz, los archivos persistentes de Slurm se guardarán en /srv/bishop/slurm.

```
sudo mkdir -p /srv/bishop/slurm/{log,state,spool/slurmd}  
sudo chown -R slurm:slurm /srv/bishop/slurm  
sudo chmod -R 755 /srv/bishop/slurm
```

También crear el directorio de configuración:

```
sudo mkdir -p /etc/slurm  
sudo chown slurm:slurm /etc/slurm
```

8.11 Detectar recursos del servidor

Una vez instalado Slurm, se debe obtener la configuración real del nodo:

```
slurmd -C
```

Este comando devuelve una línea similar a:

```
NodeName=maginot CPUs=72 Boards=1 SocketsPerBoard=... CoresPerSocket=...  
ThreadsPerCore=... RealMemory=...
```

Esta línea debe usarse como base para configurar el nodo en `slurm.conf`.

8.12 Crear slurm.conf

Crear el archivo principal de configuración:

```

sudo tee /etc/slurm/slurm.conf > /dev/null <<'EOF'
ClusterName=maginot
SlurmctldHost=maginot
SlurmUser=slurm
AuthType=auth/munge

StateSaveLocation=/srv/bishop/slurm/state
SlurmdSpoolDir=/srv/bishop/slurm/spool/slurmd

SlurmctldPidFile=/run/slurmctld.pid
SlurmdPidFile=/run/slurmd.pid

SlurmctldLogFile=/srv/bishop/slurm/log/slurmctld.log
SlurmdLogFile=/srv/bishop/slurm/log/slurmd.log

SwitchType=switch/none
MpiDefault=none
ProctrackType=proctrack/cgroup
TaskPlugin=task/cgroup,task/affinity
ReturnToService=2
SchedulerType=sched/backfill
SelectType=select/cons_tres
SelectTypeParameters=CR_Core

JobCompType=jobcomp/filetxt
JobCompLoc=/srv/bishop/slurm/log/job_completions.log

NodeName=maginot CPUs=72 Boards=1 SocketsPerBoard=2 CoresPerSocket=18
    ThreadsPerCore=2 RealMemory=257303 State=UNKNOWN
PartitionName=ripley Nodes=maginot Default=YES MaxTime=INFINITE State=UP
EOF

```

La línea NodeName debe ajustarse con la salida real de:

```
slurmd -C
```

8.13 Crear cgroup.conf

Rocky Linux 10 usa cgroups v2, por lo que se crea el archivo:

```

sudo tee /etc/slurm/cgroup.conf > /dev/null <<'EOF'
CgroupPlugin=cgroup/v2
EnableControllers=yes
EOF

```

Ajustar permisos:

```
sudo chown slurm:slurm /etc/slurm/slurm.conf /etc/slurm/cgroup.conf
sudo chmod 644 /etc/slurm/slurm.conf /etc/slurm/cgroup.conf
sudo setfacl -m u:slurm:x /srv/bishop
```

8.14 Crear archivos de log

```
sudo touch /srv/bishop/slurm/log/slurmctld.log
sudo touch /srv/bishop/slurm/log/slurmd.log
sudo touch /srv/bishop/slurm/log/job_completions.log

sudo chown slurm:slurm /srv/bishop/slurm/log/*.log
```

8.15 Iniciar servicios

```
sudo systemctl enable --now munge
sudo systemctl enable --now slurmctld
sudo systemctl enable --now slurmd
```

Si ya estaban iniciados:

```
sudo systemctl restart munge
sudo systemctl restart slurmctld
sudo systemctl restart slurmd
```

8.16 Verificar instalación

Revisar el estado de los servicios:

```
systemctl status munge --no-pager -l
systemctl status slurmctld --no-pager -l
systemctl status slurmd --no-pager -l
```

Ver la partición:

```
sinfo
```

La salida esperada debe mostrar algo similar a:

```
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
ripley*    up    infinite    1    idle maginot
```

Revisar el nodo:

```
scontrol show node maginot
```

8.17 Probar ejecución con srun

```
srun -p ripleY -N1 -n1 hostname
```

La salida esperada es:

```
maginot
```

8.18 Probar ejecución con sbatch

Crear un script de prueba:

```
cat > ~/prueba_slurm.sh <<'EOF'
#!/bin/bash
#SBATCH -J prueba
#SBATCH -p ripleY
#SBATCH -N 1
#SBATCH -n 1
#SBATCH -c 2

echo "Inicio: $(date)"
echo "Host: $(hostname)"
echo "Job ID: $SLURM_JOB_ID"
echo "CPUs asignadas: $SLURM_CPUS_PER_TASK"

sleep 10

echo "Fin: $(date)"
EOF
```

Enviar el trabajo:

```
sbatch ~/prueba_slurm.sh
```

Ver la cola:

```
squeue
```

Revisar la salida:

```
cat slurm-*.out
```

8.19 Probar encolamiento de trabajos

Para verificar que Slurm administra correctamente los recursos, se pueden enviar más trabajos de los que caben simultáneamente.

Crear script:

```
cat > ~/testCola_maginot.sh <<'EOF'
#!/bin/bash
#SBATCH -J cola_maginot
#SBATCH -p riple
#SBATCH -N 1
#SBATCH -n 1
#SBATCH -c 8
#SBATCH --time=00:05:00

echo "JobID: $SLURM_JOB_ID"
echo "Host: $(hostname)"
echo "Inicio: $(date)"
echo "CPUs asignadas: $SLURM_CPUS_PER_TASK"

sleep 120

echo "Fin: $(date)"
EOF
```

Enviar 20 trabajos:

```
for i in {1..20}; do
    sbatch ~/testCola_maginot.sh
done
```

Ver la cola:

```
squeue
```

También se puede observar en vivo:

```
watch -n 2 squeue
```

Salir de watch con:

```
Ctrl + C
```

8.20 Revisar registros de trabajos completados

Como se configuró:

```
JobCompType=jobcomp/filetxt  
JobCompLoc=/srv/bishop/slurm/log/job_completions.log
```

los trabajos completados pueden revisarse con:

```
tail -n 20 /srv/bishop/slurm/log/job_completions.log
```

Para seguir el registro en vivo:

```
sudo tail -f /srv/bishop/slurm/log/job_completions.log
```

8.21 Notas sobre sacct

En esta instalación no se configuró slurmdbd, por lo que el comando:

```
sacct
```

puede mostrar:

```
Slurm accounting storage is disabled
```

Esto no indica un error en la instalación básica. Significa que no se ha configurado el sistema completo de contabilidad de Slurm con base de datos.

Para una instalación inicial en una sola máquina, el registro con jobcomp/filetxt puede ser suficiente.

8.22 Comandos útiles para usuarios

Ver particiones disponibles:

```
sinfo
```

Ver trabajos en cola:

```
squeue
```

Enviar un trabajo:

```
sbatch script.sh
```

Cancelar un trabajo:

```
scancel JOBID
```

Ejecutar un comando interactivo sencillo:

```
srun -p riplew -N1 -n1 hostname
```

8.23 Comandos útiles para administración

Reiniciar servicios:

```
sudo systemctl restart slurmctld  
sudo systemctl restart slurmd
```

Releer configuración:

```
sudo scontrol reconfigure
```

Ver estado del nodo:

```
scontrol show node maginot
```

Ver logs del controlador:

```
sudo tail -f /srv/bishop/slurm/log/slurmctld.log
```

Ver logs del nodo:

```
sudo tail -f /srv/bishop/slurm/log/slurmd.log
```

Ver trabajos completados:

```
sudo tail -f /srv/bishop/slurm/log/job_completions.log
```

8.24 Conclusión

Con esta configuración, Slurm queda funcionando en una sola máquina con Rocky Linux 10.1. Aunque no se trata de un clúster multinodo, esta instalación permite controlar el uso de CPUs, organizar trabajos en cola y registrar ejecuciones.

Este esquema es útil para servidores compartidos de bioinformática, donde distintos usuarios ejecutan análisis que pueden consumir muchos recursos. Al obligar a enviar trabajos mediante Slurm, se reduce el riesgo de saturar el servidor y se promueve una forma más ordenada y reproducible de trabajar.